

claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf\_416cc0e5-a35\agent-a53750a3de4534cc6.jsonl`
- SHA-256: `ea281a4c2de089deb1af366fc96c2ed858a1edccbcf5819453f818c68977b9d0`
- Source modified: `2026-06-22T09:45:07+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf\_416cc0e5-a35`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`

Transcript

1. user (2026-06-22T09:42:12.650Z)

Review the NIGHTLY REEL-COUNT engine-regression change under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e (a NEW, additive PR). It runs the configured WASB pipeline on a GCP GPU VM nightly, asserts reel-count (==) + quality metrics + cost, emails a report, self-deletes. Files: scripts/nightly_reelcount.py, scripts/nightly_email.py, deploy/gcp/nightly_regression/run_on_vm.sh, deploy/gcp/nightly_regression/launch.sh, deploy/gcp/nightly_regression/register_scheduler.sh, deploy/gcp/nightly_regression/Dockerfile.launcher, eval_baselines/reelcount_manifest.json. Read them under that absolute path. Cite file:line. Report only REAL defects (correctness/safety/security that would bite in production or a regression that escapes), not style. Return raw structured findings.

LENS: INTEGRATION FAITHFULNESS ? does the runner match how the code ACTUALLY works? Verify against backend/ under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e: (a) segmenter_registry.get(name) returns a CLASS constructed as cls(db, config) with .process_video(video_id, path) returning Result[List[dict]] ? and the returned dicts' keys (start_time/end_time?) match _segments_to_intervals; (b) backend.storage.fetch(uri) signature + that it handles gs://; (c) backend.eval.rally_seg_eval.score_intervals(preds, gts) returns the keys the runner reads (tol_f1/f1@0.5/f1@0.25); (d) backend.config.load_config() exists + the dotted overrides target real fields (indexing.default_segmenter etc.); (e) Database(db_path) + the reel count via len(segs). Flag any API mismatch that would crash or silently no-op on the real VM.

2. assistant (2026-06-22T09:42:14.861Z)

I'll analyze this nightly regression change thoroughly, checking integration faithfulness against the actual backend implementation. Let me start by reading the target files.

3. user (2026-06-22T09:42:15.988Z)

```
1      """Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM
nightly).
2
3      Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a
deployment serves
4      it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does
not change**
5      vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score
of CACHED
6      trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real
model*, so it
7      needs a GPU ? hence it runs on an ephemeral GCP GPU VM
(``deploy/gcp/nightly_regression/``), emails the
8      owner the report, and self-deletes. It is **default-additive**: a tripwire, not the
promotion gate.
```

```

9
10     What it asserts / reports, per the owner's ask ("quality metrics + regression report +
cost"):
11     ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected
highlight clips.
12     Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so
this is asserted
13     **EXACT** (with a re-run-once guard for the one known flake: a transient DB-insert-
None, see $guard).
14     ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match
block from
15     ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small
tolerance.
16     ? COST of the run ? VM uptime (or processing wall) × machine $/hr via
``backend.billing`` ? USD + INR.
17
18     Extensible pattern (add a video = one manifest row + re-freeze):
19     ``eval_baselines/reelcount_manifest.json`` ? the videos (gs:// / local / gdrive://
URIs) + segmenter
20     ``eval_baselines/reelcount_baseline.json`` ? the frozen expected counts (regenerated,
never hand-edited)
21
22     Usage
23     -----
24     Freeze the baseline from the current configured+checkpointed engine (after an intended
change, or first run):
25     python scripts/nightly_reelcount.py --update-baseline
26
27     Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails
if --email):
28     python scripts/nightly_reelcount.py --email
29
30     Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality
dropped) · 2 operational
31     (manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter
changed ? re-freeze).
32     """
33     from __future__ import annotations
34
35     import argparse
36     import datetime as _dt
37     import json
38     import os
39     import sys
40     from dataclasses import dataclass, field
41     from typing import Any, Dict, List, Optional, Tuple
42
43     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
44     if REPO_ROOT not in sys.path:
45         sys.path.insert(0, REPO_ROOT)
46
47     # billing is pure (typing only) ? safe to import at module top.
48     from backend import billing # noqa: E402
49
50     DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
51     DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")
52     DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")
53
54     #: Quality metrics we report + (when labels exist) gate ? higher is better, small
tolerance.
55     QUALITY_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
56     DEFAULT_QUALITY_TOL = 0.02 # quality is float/labeled ? a looser tripwire than the
exact reel-count
57     DEFAULT_FX_INR_PER_USD = 83.0
58     #: Fallback machine price if the manifest doesn't pin one (T4 on n1-standard-8, asia-

```

```

south1, ~mid-2026).
59     DEFAULT_MACHINE_USD_PER_HR = 0.35
60
61
62     @dataclass(frozen=True)
63     class Tolerances:
64         quality_tol: float = DEFAULT_QUALITY_TOL
65
66         def to_dict(self) -> Dict[str, Any]:
67             return {"quality_tol": self.quality_tol}
68
69
70     # ----- #
71     # Pure cost math (wraps backend.billing ? no IO).
72     # ----- #
73     def cost_report(billed_seconds: float, machine_usd_per_hr: float,
74                     fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
75         """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing``
76         arithmetic).
77         ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes
78         ``--vm-uptime-seconds``;
79         otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays
80         for boot/install)."""
81         rate_per_sec = float(machine_usd_per_hr) / 3600.0
82         usage = {billing.WALL_SECONDS: float(billed_seconds)}
83         rates = {billing.WALL_SECONDS: rate_per_sec}
84         usd, breakdown = billing.compute_cost_usd(usage, rates)
85         return {
86             "billed_seconds": round(float(billed_seconds), 1),
87             "gpu_seconds": round(float(gpu_seconds), 1),
88             "machine_usd_per_hr": float(machine_usd_per_hr),
89             "usd": usd,
90             "inr": billing.to_inr(usd, fx_inr_per_usd),
91             "breakdown": breakdown,
92         }
93
94     # ----- #
95     # Pure summary + comparison core (no IO ? unit-testable with plain dicts).
96     # ----- #
97     def summarize_results(run_results: List[Dict[str, Any]], segmenter: str,
98                           cost: Dict[str, Any]) -> Dict[str, Any]:
99         """Compact, comparable snapshot from per-video run results.
100         Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}|None,
101         wall_seconds}``.
102         A video that errored keeps ``reel_count=None`` + ``ok=False`` (so the diff flags it,
103         never hides it)."""
104         per_video: Dict[str, Any] = {}
105         for r in run_results:
106             per_video[r["name"]] = {
107                 "reel_count": r.get("reel_count"),
108                 "ok": bool(r.get("ok", False)),
109                 "error": r.get("error"),
110                 "quality": r.get("quality"),
111             }
112         return {"segmenter": segmenter, "cost": cost, "per_video": per_video,
113                 "n": len(per_video)}
114
115     @dataclass
116     class Finding:
117         video: str
118         metric: str
119         # "reel_count" | "error" | a quality key | "segmenter" |

```

```

"corpus"
118     kind: str                # "regression" | "improvement" | "stale"
119     baseline: Optional[float] = None
120     current: Optional[float] = None
121     detail: str = ""
122
123     def message(self) -> str:
124         if self.kind == "stale":
125             return f"[STALE] {self.metric}: {self.detail}"
126         if self.metric == "error":
127             return f"[REGRESSION] {self.video}: pipeline ERROR ? {self.detail}"
128         b = "?" if self.baseline is None else f"{self.baseline:g}"
129         c = "?" if self.current is None else f"{self.current:g}"
130         tag = "REGRESSION" if self.kind == "regression" else "improvement"
131         return f"[{tag}] {self.video}/{self.metric}: {b} ? {c}"
132
133
134     @dataclass
135     class NightlyResult:
136         findings: List[Finding] = field(default_factory=list)
137         added: List[str] = field(default_factory=list)
138         removed: List[str] = field(default_factory=list)
139
140         @property
141         def regressions(self) -> List[Finding]:
142             return [f for f in self.findings if f.kind == "regression"]
143
144         @property
145         def improvements(self) -> List[Finding]:
146             return [f for f in self.findings if f.kind == "improvement"]
147
148         @property
149         def stale(self) -> List[Finding]:
150             return [f for f in self.findings if f.kind == "stale"]
151
152         def overall_status(self) -> str:
153             if self.regressions:
154                 return "REGRESSION"
155             if self.stale:
156                 return "STALE"
157             return "PASS"
158
159         def exit_code(self) -> int:
160             if self.regressions:
161                 return 1
162             if self.stale:
163                 return 4
164             return 0
165
166
167     def compare(baseline: Dict[str, Any], current: Dict[str, Any], tols: Tolerances) ->
NightlyResult:
168         """Diff current run vs frozen baseline.
169
170         * Segmenter changed ? STALE (numbers not comparable; re-freeze).
171         * Corpus (video set) changed ? STALE + report added/removed; still diff the
INTERSECTION.
172         * Per shared video: reel_count EXACT (any change = regression); a pipeline ERROR =
regression;
173         quality metrics drop beyond ``quality_tol`` = regression (rise = improvement).
174         """
175         res = NightlyResult()
176         if baseline.get("segmenter") != current.get("segmenter"):
177             res.findings.append(Finding("", "segmenter", "stale",
178                                     detail=f"{baseline.get('segmenter')} ?

```

```

{current.get('segmenter')}}))
179         return res
180
181     b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
182     res.added = sorted(set(c_pv) - set(b_pv))
183     res.removed = sorted(set(b_pv) - set(c_pv))
184     if res.added or res.removed:
185         res.findings.append(Finding("", "corpus", "stale",
186                                     detail=f"added={res.added} removed={res.removed}"))
187
188     for v in sorted(set(b_pv) & set(c_pv)):
189         b, c = b_pv[v], c_pv[v]
190         # 1) pipeline must still succeed
191         if not c.get("ok", False) or c.get("reel_count") is None:
192             res.findings.append(Finding(v, "error", "regression",
193                                         detail=str(c.get("error") or "no reel_count
194 produced"))))
195             continue
196         # 2) reel count ? EXACT
197         bc, cc = b.get("reel_count"), c.get("reel_count")
198         if bc is not None and cc is not None and cc != bc:
199             res.findings.append(Finding(v, "reel_count", "regression", float(bc),
200 float(cc)))
201         # 3) quality metrics ? tolerance (only if both sides have them)
202         bq, cq = b.get("quality") or {}, c.get("quality") or {}
203         for m in QUALITY_HIGHER:
204             bv, cv = bq.get(m), cq.get(m)
205             if bv is None or cv is None:
206                 continue
207             if cv < bv - tols.quality_tol:
208                 res.findings.append(Finding(v, m, "regression", bv, cv))
209             elif cv > bv + tols.quality_tol:
210                 res.findings.append(Finding(v, m, "improvement", bv, cv))
211
212     return res
213
214 # ----- #
215 # Report formatting (pure).
216 # ----- #
217 def _q(qd: Optional[Dict[str, Any]], key: str) -> str:
218     if not qd or qd.get(key) is None:
219         return "?"
220     return f"{qd[key]:.3f}"
221
222 def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
223                  result: NightlyResult, date: str, head: str) -> str:
224     status = result.overall_status()
225     badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION",
226             "STALE": "? STALE (re-freeze the baseline)"}[status]
227     cost = current.get("cost", {})
228     L: List[str] = [
229         f"# Nightly engine regression (reel-count + quality + cost) ? {date}",
230         "",
231         f"***Status: {badge}** · exit code {result.exit_code()} · segmenter
232 {current.get('segmenter')}",
233         "",
234         f"- HEAD {head} · baseline {baseline.get('git_commit', '?')} (frozen
235 {baseline.get('generated_at', '?')})",
236         f"- **Run cost: ${cost.get('usd', 0):.4f} ({cost.get('inr', 0):.2f})** · "
237         f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr',
238 0)}/hr",
239         "",
240     ]
241     if result.regressions:

```

```

238         L += ["## ? Regressions", ""] + [f"- {f.message()}" for f in result.regressions]
+ [""]
239     if result.stale:
240         L += ["## ? Stale / not comparable", ""] + [f"- {f.message()}" for f in
result.stale]
241         L += ["- Action: `python scripts/nightly_reelcount.py --update-baseline` (after
confirming the change is intended).", ""]
242
243     # Per-video table.
244     b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
245     L += ["## Per-video", "",
246         "| video | reels (base?now) | tol_f1 (base?now) | f1@0.5 | status |",
247         "|---|---|---|---|---|"]
248     for v in sorted(set(b_pv) | set(c_pv)):
249         b, c = b_pv.get(v, {}), c_pv.get(v, {})
250         bc = b.get("reel_count")
251         cc = c.get("reel_count")
252         rc = f"'?' if bc is None else bc?{'ERROR' if not c.get('ok', True) else ('?'
if cc is None else cc)}"
253         bq, cq = b.get("quality"), c.get("quality")
254         tf = f"{_q(bq, 'tol_f1')}?{_q(cq, 'tol_f1')}"
255         f5 = f"{_q(bq, 'f1@0.5')}?{_q(cq, 'f1@0.5')}"
256         vstatus = "?" if any(f.video == v and f.kind == "regression" for f in
result.findings) else "?"
257         L.append(f"| {v} | {rc} | {tf} | {f5} | {vstatus} |")
258     L.append("")
259
260     if result.improvements:
261         L += ["_Improvements over baseline (re-freeze to ratchet up):_"] \
+ [f"- {f.message()}" for f in result.improvements] + [""]
262     L += ["---",
263         "_Full configured+checkpointed pipeline on the real model (GCP GPU VM).
Tripwire only ? does "
264         "not change the promotion gate. Complements PR #202's cached-trajectory CPU
re-score._"]
265     return "\n".join(L)
266
267
268
269     # ----- #
270     # IO shell ? running the configured pipeline per video (needs backend + GPU + video).
271     # ----- #
272     def _load_gts(labels_ref: Optional[str]) -> Optional[List[Tuple[float, float]]]:
273         """Load ground-truth rally intervals from a golden-features JSON (`gts` key) or a
rallies CSV
274         (`start,end` columns / pairs). Returns None when no labels are configured."""
275         if not labels_ref:
276             return None
277         from backend.storage import fetch
278         local = fetch(labels_ref)
279         if local.endswith(".json"):
280             with open(local, encoding="utf-8") as fh:
281                 data = json.load(fh)
282                 return [(float(s), float(e)) for s, e in (data.get("gts") or [])]
283         # CSV: header start,end (or first two numeric columns)
284         import csv
285         out: List[Tuple[float, float]] = []
286         with open(local, encoding="utf-8") as fh:
287             for row in csv.reader(fh):
288                 try:
289                     out.append((float(row[0]), float(row[1])))
290                 except (ValueError, IndexError):
291                     continue
292         return out
293
294

```

```

295     def _segments_to_intervals(segs: List[Dict[str, Any]]) -> List[Tuple[float, float]]:
296         """(start, end) pairs from segmenter result dicts, tolerant of the two key
conventions in the
297         codebase (``start_time``/``end_time`` ? motion/DB shape ? or ``start``/``end``).
Used only for the
298         optional quality metrics; the reel COUNT is ``len(segs)`` regardless, so a key
mismatch degrades
299         quality scoring gracefully (skipped) without ever miscounting reels."""
300         out: List[Tuple[float, float]] = []
301         for s in segs:
302             start = s.get("start_time", s.get("start"))
303             end = s.get("end_time", s.get("end"))
304             if start is not None and end is not None:
305                 out.append((float(start), float(end)))
306         return out
307
308
309     def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
310                     rerun_on_mismatch: bool = True,
311                     baseline_count: Optional[int] = None) -> Dict[str, Any]:
312         """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy
backend imports.
313
314         The re-run-once guard (the one known non-determinism ? a transient
``add_play_segment``?None drop,
315         database.py): if the count != the frozen baseline, re-process the video ONCE; a
transient drop won't
316         reproduce, a real change will."""
317         import random
318         import time
319
320         from backend.eval import rally_seg_eval
321         from backend.infrastructure.database import Database
322         from backend.pipeline.segmenters import segmenter_registry
323         from backend.results import Failure
324         from backend.storage import fetch
325         from backend.utils.hashing import compute_video_id
326
327         name = video["name"]
328         try:
329             local = fetch(video["uri"])
330         except Exception as e: # noqa: BLE001 ? surface, never hide
331             return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed:
{e}",
332                     "quality": None, "wall_seconds": 0.0}
333
334         gts = _load_gts(video.get("labels_uri"))
335         seg_cls = segmenter_registry.get(segmenter)
336         if not seg_cls:
337             return {"name": name, "reel_count": None, "ok": False,
338                     "error": f"segmenter '{segmenter}' not registered", "quality": None,
"wall_seconds": 0.0}
339
340     def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]], float, Optional[str]]:
341         random.seed(0) # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
342         db = Database(db_path)
343         video_id = compute_video_id(local)
344         t0 = time.monotonic()
345         result = seg_cls(db, config).process_video(video_id, local)
346         wall = time.monotonic() - t0
347         if isinstance(result, Failure):
348             return None, None, wall, getattr(result, "message", "segmenter failed")
349         segs = result.value if hasattr(result, "value") else result

```

```

350         intervals = _segments_to_intervals(segs)
351         return len(segs), intervals, wall, None
352
353     import tempfile
354     total_wall = 0.0
355     with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
356         count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
357         total_wall += wall
358         if err is not None:
359             return {"name": name, "reel_count": None, "ok": False, "error": err,
360                    "quality": None, "wall_seconds": round(total_wall, 2)}
361         # re-run-once guard for the transient DB-drop flake
362         if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
363             count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
364             total_wall += wall2
365             if err2 is None and count2 is not None:
366                 count, intervals = count2, intervals2 # the reproduced (or corrected)
367
368         value
369         quality = None
370         if gts and intervals is not None:
371             row = rally_seg_eval.score_intervals(intervals, gts)
372             quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
373 "tol_precision", "tol_recall")}
374             if k in row}
375
376         return {"name": name, "reel_count": count, "ok": True, "error": None,
377                "quality": quality, "wall_seconds": round(total_wall, 2)}
378
379     def load_manifest(path: str) -> Dict[str, Any]:
380         with open(path, encoding="utf-8") as fh:
381             return json.load(fh)
382
383     def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
384                   rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
385         """Run every manifest video through the configured pipeline. Returns (run_results,
386 total_wall_s)."""
387         from backend.config import load_config
388
389         config = load_config()
390         # apply manifest config overrides onto the typed config (dotted keys)
391         for dotted, val in (manifest.get("config_overrides") or {}).items():
392             _set_dotted(config, dotted, val)
393         segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
394 "default_segmenter", "motion")
395         b_pv = (baseline or {}).get("per_video", {})
396         results: List[Dict[str, Any]] = []
397         total_wall = 0.0
398         for v in manifest.get("videos", []):
399             base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
400             r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
401             total_wall += float(r.get("wall_seconds") or 0.0)
402             results.append(r)
403         return results, total_wall
404
405     def _set_dotted(config: Any, dotted: str, value: Any) -> None:
406         """Best-effort dotted-path set onto a pydantic config (e.g.
407 'indexing.motion_threshold')."""
408         parts = dotted.split(".")
409         obj = config
410         for p in parts[:-1]:
411             obj = getattr(obj, p, None)

```



```

410         if obj is None:
411             return
412     try:
413         setattr(obj, parts[-1], value)
414     except Exception: # noqa: BLE001 ? overrides are best-effort; a bad key shouldn't
crash the run
415         pass
416
417
418     def _git_head() -> str:
419         import subprocess
420         try:
421             out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
422                                 capture_output=True, text=True, timeout=10)
423             return out.stdout.strip() or "?"
424         except Exception: # noqa: BLE001
425             return "?"
426
427
428     def load_baseline(path: str) -> Optional[Dict[str, Any]]:
429         if not os.path.exists(path):
430             return None
431         with open(path, encoding="utf-8") as fh:
432             return json.load(fh)
433
434
435     def save_baseline(path: str, snapshot: Dict[str, Any]) -> None:
436         out = dict(snapshot)
437         out["generated_at"] = _dt.date.today().isoformat()
438         out["git_commit"] = _git_head()
439         out["_doc"] = ("Frozen nightly reel-count baseline. Regenerate with `python
scripts/nightly_reelcount.py "
440                        "--update-baseline` after an INTENDED engine change or when adding a
video. Tied to the "
441                        "configured+checkpointed engine + the manifest corpus.")
442         out.pop("cost", None) # cost is per-run, not part of the frozen baseline
443         os.makedirs(os.path.dirname(path), exist_ok=True)
444         tmp = path + ".tmp"
445         with open(tmp, "w", encoding="utf-8") as fh:
446             json.dump(out, fh, indent=2, sort_keys=True)
447             fh.write("\n")
448         os.replace(tmp, path)
449
450
451     def build_parser() -> argparse.ArgumentParser:
452         p = argparse.ArgumentParser(description="Nightly E2E engine regression: reel-count +
quality + cost.")
453         p.add_argument("--manifest", default=DEFAULT_MANIFEST)
454         p.add_argument("--baseline", default=DEFAULT_BASELINE)
455         p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR)
456         p.add_argument("--quality-tol", type=float, default=DEFAULT_QUALITY_TOL)
457         p.add_argument("--vm-uptime-seconds", type=float, default=None,
458                        help="authoritative VM uptime (boot?delete) for the cost line; "
459                        "defaults to the in-process pipeline wall time (a lower bound)")
460         p.add_argument("--no-rerun-guard", action="store_true",
461                        help="disable the re-run-once-on-mismatch guard (the DB-drop flake
mitigation)")
462         p.add_argument("--update-baseline", action="store_true",
463                        help="freeze the current reel counts + quality as the new baseline
instead of checking")
464         p.add_argument("--email", action="store_true", help="email the report (see
scripts/nightly_email.py)")
465         p.add_argument("--email-to", default=None, help="override the report recipient")
466         p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
for tests)")

```

```

467         p.add_argument("--no-report", action="store_true", help="skip writing the dated
report file")
468         return p
469
470
471     def main(argv: Optional[List[str]] = None) -> int:
472         import logging
473         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
474         log = logging.getLogger("nightly_reelcount")
475         args = build_parser().parse_args(argv)
476         tols = Tolerances(quality_tol=args.quality_tol)
477         date = args.date or _dt.date.today().isoformat()
478
479         if not os.path.exists(args.manifest):
480             log.error("manifest not found: %s", args.manifest)
481             return 2
482         manifest = load_manifest(args.manifest)
483         baseline = load_baseline(args.baseline)
484
485         try:
486             run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not
args.no_rerun_guard)
487             except Exception as e: # noqa: BLE001 ? operational failure (deps/config) ? exit 2,
never a false "pass"
488                 log.error("run failed: %s", e)
489                 return 2
490             if not run_results:
491                 log.error("no videos in manifest %s", args.manifest)
492                 return 2
493
494             billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else
total_wall
495             cost = cost_report(
496                 billed_seconds=billed,
497                 machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
DEFAULT_MACHINE_USD_PER_HR)),
498                 fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
499                 gpu_seconds=total_wall,
500             )
501             segmenter = manifest.get("segmenter", "motion")
502             snapshot = summarize_results(run_results, segmenter, cost)
503             snapshot["git_commit"] = _git_head()
504             snapshot["generated_at"] = date
505
506             if args.update_baseline:
507                 save_baseline(args.baseline, snapshot)
508                 log.info("baseline frozen ? %s", args.baseline)
509                 for r in run_results:
510                     log.info(" %s: reels=%s ok=%s", r["name"], r.get("reel_count"),
r.get("ok"))
511                 return 0
512
513             if baseline is None:
514                 log.error("no baseline at %s ? freeze one with --update-baseline (on the
configured engine).", args.baseline)
515                 return 3
516
517             result = compare(baseline, snapshot, tols)
518             report = format_report(baseline, snapshot, result, date, snapshot["git_commit"])
519
520             if not args.no_report:
521                 os.makedirs(args.report_dir, exist_ok=True)
522                 md_path = os.path.join(args.report_dir, f"{date}.md")
523                 with open(md_path, "w", encoding="utf-8") as fh:
524                     fh.write(report + "\n")

```

```

525         with open(os.path.join(args.report_dir, f"{date}.json"), "w", encoding="utf-8")
as fh:
526             json.dump({"date": date, "status": result.overall_status(), "exit_code":
result.exit_code(),
527                         "snapshot": snapshot, "regressions": [f.message() for f in
result.regressions]},
528                         fh, indent=2, sort_keys=True)
529             log.info("report ? %s", md_path)
530
531         print(report)
532
533         if args.email:
534             try:
535                 from scripts.nightly_email import send_report
536                 subject = f"[KhelSutra nightly] {result.overall_status()} ? reel-count
engine regression {date}"
537                 sent = send_report(subject, report, to=args.email_to)
538                 log.info("email %s", "sent" if sent else "skipped (no creds ? report still
written)")
539             except Exception as e: # noqa: BLE001 ? a broken mailer must NOT mask the
regression verdict
540                 log.error("email failed (report still written): %s", e)
541
542         code = result.exit_code()
543         if code == 1:
544             log.error("REEL-COUNT REGRESSION (%d) ? see %s", len(result.regressions),
args.report_dir)
545         elif code == 4:
546             log.warning("baseline STALE ? corpus/segmenter changed; re-freeze with --update-
baseline")
547         else:
548             log.info("nightly reel-count check PASSED (cost $%.4f / ?%.2f)", cost["usd"],
cost["inr"])
549         return code
550
551
552     if __name__ == "__main__":
553         raise SystemExit(main())
554

```

4. user (2026-06-22T09:42:16.199Z)

```

1     """Email the nightly regression report ? stdlib SMTP, creds from env or GCP Secret
Manager.
2
3     Used by ``scripts/nightly_reelcount.py --email`` (and reusable by any nightly).
Deliberately
4     dependency-light (stdlib ``smtplib`` + ``email.message``) and **graceful**: if no SMTP
password can
5     be resolved it logs + returns ``False`` (the report is still written/printed ? a broken
mailer must
6     never mask a regression).
7
8     Credential resolution (first hit wins), so it works on a GCP VM, locally, or in tests:
9     ? Recipient: ``--email-to`` arg ? ``NIGHTLY_EMAIL_TO`` ? ``avi.dullu@gmail.com``
10    ? SMTP host: ``NIGHTLY_SMTP_HOST`` (default ``smtp.gmail.com``) . port
``NIGHTLY_SMTP_PORT`` (587, STARTTLS)
11    ? SMTP user: ``NIGHTLY_SMTP_USER`` (default = recipient)
12    ? SMTP pass: ``NIGHTLY_SMTP_PASS`` ? Secret Manager ``NIGHTLY_SMTP_SECRET``
13    (``projects/<p>/secrets/<name>/versions/latest``; via the google-cloud-
secret-manager
14    lib if installed, else the ``gcloud`` CLI). A Gmail **app password** is
the simplest.
15
16    Set the secret once (owner): echo -n '<app-password>' | gcloud secrets create nightly-

```

```

smtp-pass --data-file=-
17     and on the VM: export NIGHTLY SMTP_SECRET=projects/<proj>/secrets/nightly-smtp-
pass/versions/latest
18         export NIGHTLY SMTP_USER=<your-gmail>
NIGHTLY_EMAIL_TO=avi.dullu@gmail.com
19     """
20     from __future__ import annotations
21
22     import logging
23     import os
24     from email.message import EmailMessage
25     from typing import Optional
26
27     log = logging.getLogger("nightly_email")
28
29     DEFAULT_TO = "avi.dullu@gmail.com"
30     DEFAULT_HOST = "smtp.gmail.com"
31     DEFAULT_PORT = 587
32
33
34     def build_message(subject: str, body_md: str, from_addr: str, to_addr: str) ->
EmailMessage:
35         """A text/plain email carrying the markdown report (renders fine in any client).
Pure ? testable."""
36         msg = EmailMessage()
37         msg["Subject"] = subject
38         msg["From"] = from_addr
39         msg["To"] = to_addr
40         msg.set_content(body_md)
41         return msg
42
43
44     def resolve_secret(env: Optional[dict] = None) -> Optional[str]:
45         """Resolve the SMTP password from env or GCP Secret Manager. Returns None if
unresolvable."""
46         env = env if env is not None else os.environ
47         direct = env.get("NIGHTLY SMTP_PASS")
48         if direct:
49             return direct
50         secret_ref = env.get("NIGHTLY SMTP_SECRET")
51         if not secret_ref:
52             return None
53         # Prefer the client lib; fall back to the gcloud CLI (no extra dependency required).
54         try:
55             from google.cloud import secretmanager # type: ignore
56             client = secretmanager.SecretManagerServiceClient()
57             return
client.access_secret_version(name=secret_ref).payload.data.decode("utf-8")
58         except Exception: # noqa: BLE001 ? lib absent or auth issue ? try the CLI
59             pass
60         try:
61             import subprocess
62             name = secret_ref.split("/secrets/", 1)[1].split("/", 1)[0] if "/secrets/" in
secret_ref else secret_ref
63             out = subprocess.run(["gcloud", "secrets", "versions", "access", "latest", "--
secret", name],
64                                 capture_output=True, text=True, timeout=30)
65             return out.stdout.strip() if out.returncode == 0 and out.stdout.strip() else
None
66         except Exception as e: # noqa: BLE001
67             log.warning("secret-manager resolution failed: %s", e)
68             return None
69
70
71     def send_report(subject: str, body_md: str, to: Optional[str] = None,

```

```

72         env: Optional[dict] = None) -> bool:
73         """Send the report. Returns True if sent, False if skipped (no creds) ? never raises
on missing creds."""
74         env = env if env is not None else os.environ
75         to_addr = to or env.get("NIGHTLY_EMAIL_TO") or DEFAULT_TO
76         host = env.get("NIGHTLY_SMTP_HOST", DEFAULT_HOST)
77         port = int(env.get("NIGHTLY_SMTP_PORT", DEFAULT_PORT))
78         user = env.get("NIGHTLY_SMTP_USER") or to_addr
79         password = resolve_secret(env)
80         if not password:
81             log.warning("no SMTP password (set NIGHTLY_SMTP_PASS or NIGHTLY_SMTP_SECRET) ?
email skipped.")
82             return False
83
84         import smtplib
85         msg = build_message(subject, body_md, from_addr=user, to_addr=to_addr)
86         with smtplib.SMTP(host, port, timeout=60) as s:
87             s.starttls()
88             s.login(user, password)
89             s.send_message(msg)
90         log.info("emailed report to %s", to_addr)
91         return True
92

```

5. user (2026-06-22T09:42:17.143Z)

```

1      #!/usr/bin/env bash
2      # Startup-script for the NIGHTLY engine-regression VM. Runs UNATTENDED on boot, then
DELETES the VM
3      # (cost rule: a running GPU VM is the only real cost). Survives SSH drops (it's a
startup-script, not
4      # an interactive session). Reads its config from instance metadata (set by launch.sh).
5      #
6      # Flow: Ops Agent ? pull repo @ HEAD from GCS ? build venv + WASB env ? stage
weights ?
7      #          run scripts/nightly_reelcount.py on the gs:// golden clips (emails the report)
?
8      #          upload report to GCS ? SELF-DELETE (even on failure, via the EXIT trap).
9      #
10     # Metadata keys (launch.sh sets these): repo-tgz-uri, git-sha, gcs-bucket, report-
prefix,
11     # weights-uri, email (true/false). NOT verified in CI ? dry-run via launch.sh on a
real VM first.
12     set -uo pipefail
13     exec > >(tee -a /var/log/nightly-regression.log) 2>&1
14     echo "==== nightly-regression startup $(date -u +%FT%TZ) ====="
15     BOOT_EPOCH=$(date +%s)
16
17     md() { curl -s -H "Metadata-Flavor: Google"
"http://metadata.google.internal/computeMetadata/v1/$1"; }
18     NAME=$(md instance/name)
19     ZONE=$(md instance/zone | awk -F/ '{print $NF}')
20     REPO_TGZ=$(md instance/attributes/repo-tgz-uri || true)
21     GIT_SHA=$(md instance/attributes/git-sha || echo unknown)
22     BUCKET=$(md instance/attributes/gcs-bucket || true)
23     REPORT_PREFIX=$(md instance/attributes/report-prefix || echo nightly/reports)
24     WEIGHTS_URI=$(md instance/attributes/weights-uri || true)
25     DO_EMAIL=$(md instance/attributes/email || echo true)
26     SMTP_SECRET=$(md instance/attributes/smtp-secret || true)
27     SMTP_USER=$(md instance/attributes/smtp-user || true)
28     EMAIL_TO=$(md instance/attributes/email-to || echo avi.dullu@gmail.com)
29     DATE=$(date -u +%F)
30
31     # ALWAYS delete the VM on exit (success OR failure) ? never leave a billing GPU VM
behind.

```

```

32 cleanup() {
33     echo "==== self-delete $NAME ($ZONE) ====="
34     gcloud compute instances delete "$NAME" --zone="$ZONE" --quiet || \
35     echo "WARN: self-delete failed ? DELETE MANUALLY: gcloud compute instances delete
$NAME --zone=$ZONE -q"
36 }
37 trap cleanup EXIT
38
39 fail_email() { # best-effort failure alert (the report email won't have fired on an
early crash)
40     echo "RUN FAILED ? attempting a failure alert"
41     if [ "$DO_EMAIL" = "true" ] && [ -d ~/rally ]; then
42         cd ~/rally 2>/dev/null && . .venv/bin/activate 2>/dev/null && \
43         NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
44         python - <<PY || true
45     from scripts.nightly_email import send_report
46     send_report("[KhelSutra nightly] ERROR ? engine regression VM run failed ($DATE)",
47         "The nightly engine-regression run failed on the VM before producing a
report.\n"
48         "See gs://$BUCKET/$REPORT_PREFIX/$DATE/ and the VM serial
log.\nGIT_SHA=$GIT_SHA", to="$EMAIL_TO")
49     PY
50     fi
51 }
52 trap 'fail_email' ERR
53
54 # 1) Observability (Ops Agent) ? same as create_gpu_vm.sh bakes in.
55 curl -sSO https://dl.google.com/cloudagents/add-google-cloud-ops-agent-repo.sh && \
56     sudo bash add-google-cloud-ops-agent-repo.sh --also-install || echo "WARN: ops-agent
install failed (non-fatal)"
57
58 # 2) Pull the repo @ HEAD (staged to GCS by launch.sh ? no GitHub auth needed; uses the
worker SA).
59 mkdir -p ~/rally && cd ~/rally
60 gcloud storage cp "$REPO_TGZ" /tmp/rally.tgz
61 tar -xzf /tmp/rally.tgz -C ~/rally
62 echo "$GIT_SHA" > ~/rally/GIT_SHA
63
64 # 3) Build the env + native WASB (provider-agnostic DO scripts, reused on GCP per
deploy/gcp/README §4).
65 sudo bash deploy/digitalocean/mount_scratch.sh 2>/dev/null || true
66 export TMPDIR=/scratch 2>/dev/null || true
67 ./deploy/digitalocean/bootstrap.sh --gpu
68 . .venv/bin/activate
69 pip install -e '[storage-gcs]'
70 bash deploy/digitalocean/gpu_setup.sh || echo "WARN: gpu_setup (WASB env) returned
nonzero ? continuing"
71
72 # 4) Stage the WASB checkpoint (gpu_setup does NOT fetch weights ? README §4).
73 if [ -n "$WEIGHTS_URI" ]; then
74     mkdir -p ~/models/WASB-SBDT/pretrained_weights
75     gcloud storage cp "$WEIGHTS_URI" ~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar
76     export WASB_WEIGHTS=~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar
77     fi
78
79 # 5) Run the regression on the gs:// golden clips in the committed manifest. Cost = VM
uptime so far.
80 UPTIME=$(( $(date +%s) - BOOT_EPOCH ))
81 EMAIL_FLAG=""; [ "$DO_EMAIL" = "true" ] && EMAIL_FLAG="--email --email-to $EMAIL_TO"
82 set +e
83 NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
84     python scripts/nightly_reelcount.py --vm-uptime-seconds "$UPTIME" $EMAIL_FLAG

```

```

85     RUN_RC=$?
86     set -e
87     echo "runner exit code: $RUN_RC (0 pass / 1 regression / 4 stale)"
88
89     # 6) Upload the dated report to GCS for the record (alongside the email).
90     if [ -n "$BUCKET" ] && [ -d output/nightly_reelcount ]; then
91         gcloud storage cp -r "output/nightly_reelcount/$DATE."*
92         "gs://$BUCKET/$REPORT_PREFIX/$DATE/" || \
93         gcloud storage cp -r output/nightly_reelcount "gs://$BUCKET/$REPORT_PREFIX/$DATE/"
94     || true
95     fi
96
97     echo "==== nightly-regression done rc=$RUN_RC $(date -u +%FT%TZ) ====="
98     trap - ERR # disable the failure-email trap; the EXIT (self-delete) trap stays
99     # the EXIT trap deletes the VM

```

6. user (2026-06-22T09:42:17.778Z)

```

1     #!/usr/bin/env bash
2     # Launch ONE nightly engine-regression run on an ephemeral GCP GPU VM, then return. The
3     VM runs
4     # run_on_vm.sh as its startup-script (regression ? email ? SELF-DELETE), so this just
5     stages the repo
6     # and kicks the VM off. Driver-agnostic: call it from Cloud Scheduler?Cloud Run job, a
7     cron on any
8     # always-on box, the owner's machine, or a Claude routine. Needs gcloud auth + the repo
9     checkout.
10    #
11    # bash deploy/gcp/nightly_regression/launch.sh # L4, stages HEAD,
12    kicks the VM
13    # RALLY_GPU=t4 RALLY_REF=$(git rev-parse HEAD) bash
14    deploy/gcp/nightly_regression/launch.sh
15    #
16    # ? Run this MANUALLY once as a dry-run (watch the serial log) before wiring a scheduler
17    ? the VM-side
18    # flow is NOT covered by CI. Tail: gcloud compute instances get-serial-port-output
19    <name> --zone=<z>.
20    set -uo pipefail
21
22    PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"
23    BUCKET="${RALLY_BUCKET:-kshelutra-rally-corpus}"
24    SA="rally-worker@${PROJECT}.iam.gserviceaccount.com"
25    GPU="${RALLY_GPU:-l4}"
26    NAME="${RALLY_VM_NAME:-rally-nightly-$(date -u +%Y%m%d)}"
27    REF="${RALLY_REF:-HEAD}"
28    WEIGHTS_URI="${RALLY_WEIGHTS_URI:-gs://$BUCKET/weights/wasb_badminton_best.pth.tar}"
29    REPORT_PREFIX="${RALLY_REPORT_PREFIX:-nightly/reports}"
30    DO_EMAIL="${RALLY_EMAIL:-true}"
31    SMTP_SECRET="${NIGHTLY_SMTP_SECRET:-projects/$PROJECT/secrets/nightly-smtp-
32    pass/versions/latest}"
33    SMTP_USER="${NIGHTLY_SMTP_USER:-}"
34    EMAIL_TO="${NIGHTLY_EMAIL_TO:-avi.dullu@gmail.com}"
35    HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
36    STARTUP="$HERE/run_on_vm.sh"
37    export CLOUDSDK_CORE_DISABLE_PROMPTS=1
38
39    ZONES="${RALLY_GCP_ZONES:-asia-south1-a asia-south1-b asia-south1-c asia-southeast1-a
40    asia-southeast1-b asia-southeast1-c}"
41    if [ "$GPU" = "l4" ]; then MACHINE="g2-standard-4"; ACCEL=(); else
42    MACHINE="n1-standard-8"; ACCEL=(--accelerator=type=nvidia-tesla-t4,count=1); fi
43
44    # 1) Stage the repo @ REF to GCS (no GitHub auth on the VM; the worker SA can read the
45    bucket).
46    SHA=$(git rev-parse "$REF")

```

```

35     TGZ_URI="gs://$BUCKET/nightly/repo/${SHA}.tgz"
36     echo "== staging repo $SHA -> $TGZ_URI =="
37     TMP_TGZ="$(mktemp --suffix=.tgz)"
38     git archive --format=tar.gz -o "$TMP_TGZ" "$REF"
39     gcloud storage cp "$TMP_TGZ" "$TGZ_URI" --project="$PROJECT"
40     rm -f "$TMP_TGZ"
41
42     # 2) Zone-sweep create the VM (mirrors create_gpu_vm.sh) with run_on_vm.sh + the config
metadata.
43     ERR="$(mktemp)"
44     for Z in $ZONES; do
45         echo "== creating $NAME ($GPU) in $Z =="
46         if gcloud compute instances create "$NAME" --project="$PROJECT" --zone="$Z" \
47             --machine-type="$MACHINE" "${ACCEL[@]}" \
48             --maintenance-policy=TERMINATE --provisioning-model=STANDARD \
49             --image-family=common-cu129-ubuntu-2204-nvidia-580 --image-project=deeplearning-
platform-release \
50             --boot-disk-size=120GB --boot-disk-type=pd-balanced \
51             --service-account="$SA" --scopes=cloud-platform \
52             --metadata-from-file=startup-script="$STARTUP" \
53             --metadata=repo-tgz-uri="$TGZ_URI",git-sha="$SHA",gcs-bucket="$BUCKET",report-
prefix="$REPORT_PREFIX",weights-uri="$WEIGHTS_URI",email="$DO_EMAIL",smtp-
secret="$SMTP_SECRET",smtp-user="$SMTP_USER",email-to="$EMAIL_TO" \
54             2>"$ERR"; then
55             echo "? launched $NAME in $Z ? it will run the regression, email $EMAIL_TO, and
self-delete."
56             echo "    serial log: gcloud compute instances get-serial-port-output $NAME --zone=$Z
--project=$PROJECT"
57             echo "    if it doesn't self-delete: gcloud compute instances delete $NAME --zone=$Z
-q"
58             rm -f "$ERR"; exit 0
59         fi
60         if grep -qiE "STOCKOUT|does not have enough" "$ERR"; then echo "    (no $GPU capacity in
$Z ? next zone)"; continue; fi
61         echo "    create failed:"; tail -4 "$ERR"; rm -f "$ERR"; exit 1
62     done
63     echo "? no $GPU capacity in any swept zone ($ZONES). Retry later or RALLY_GPU=t4."
64     rm -f "$ERR"; exit 1
65

```

7. user (2026-06-22T09:42:18.940Z)

```

1     #!/usr/bin/env bash
2     # Wire the nightly engine-regression as a CLOUD-NATIVE, scale-to-zero job:
3     #   Cloud Scheduler (cron) ? Cloud Run JOB (the launcher image) ? launch.sh
4     #   ? ephemeral GPU VM (regression + email + SELF-DELETE).
5     # The only always-on cost is $0 (Cloud Run jobs + Scheduler are scale-to-zero; the GPU
VM is ephemeral).
6     #
7     # ? OWNER-RUN, ONCE, after a successful manual `bash
deploy/gcp/nightly_regression/launch.sh` dry-run.
8     # NOT verified in CI (no GCP/GPU here). Prereqs: GPU quota (README S2), the golden clips
+ weights in
9     # gs://<bucket>, and the SMTP secret (see README "Enable").
10    #
11    #   bash deploy/gcp/nightly_regression/register_scheduler.sh           # build + deploy
+ schedule (03:00 IST)
12    #   RALLY_CRON='0 3 * * *' RALLY_TZ='Asia/Kolkata' bash ../register_scheduler.sh
13    set -euo pipefail
14
15    PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"
16    REGION="${RALLY_RUN_REGION:-asia-south1}"
17    JOB="${RALLY_RUN_JOB:-nightly-regression-launcher}"
18    SCHED="${RALLY_SCHED_NAME:-nightly-regression}"
19    CRON="${RALLY_CRON:-0 3 * * *}"           # 03:00 daily

```



```

20     TZ="${RALLY_TZ:-Asia/Kolkata}"
21     SA="rally-worker@${PROJECT}.iam.gserviceaccount.com"
22     IMAGE="${RALLY_LAUNCHER_IMAGE:-${REGION}-docker.pkg.dev/${PROJECT}/rally/nightly-
launcher:latest}"
23     REPO_ROOT="$(git rev-parse --show-toplevel)"
24     export CLOUDSDK_CORE_DISABLE_PROMPTS=1
25
26     echo "== enable APIs (run, scheduler, build, artifact registry) =="
27     gcloud services enable run.googleapis.com cloudscheduler.googleapis.com \
28         cloudbuild.googleapis.com artifactregistry.googleapis.com --project="$PROJECT"
29
30     echo "== ensure the Artifact Registry repo 'rally' exists =="
31     gcloud artifacts repositories describe rally --location="$REGION" --project="$PROJECT"
>/dev/null 2>&1 || \
32     gcloud artifacts repositories create rally --repository-format=docker
--location="$REGION" --project="$PROJECT"
33
34     echo "== build + push the launcher image from the repo root =="
35     ( cd "$REPO_ROOT" && gcloud builds submit --project="$PROJECT" --tag "$IMAGE" \
36         --gcs-source-staging-dir="gs://${RALLY_BUILD_BUCKET:-khelsutra-rally-
corpus}/cloudbuild" \
37         -f deploy/gcp/nightly_regression/Dockerfile.launcher . )
38
39     echo "== create/update the Cloud Run JOB (launcher needs compute-admin to create the GPU
VM) =="
40     gcloud run jobs deploy "$JOB" --project="$PROJECT" --region="$REGION" \
41         --image="$IMAGE" --service-account="$SA" --max-retries=0 --task-timeout=900s \
42         --set-env-vars="RALLY_GPU=${RALLY_GPU:-
l4},RALLY_EMAIL=true,NIGHTLY_EMAIL_TO=${NIGHTLY_EMAIL_TO:-avi.dullu@gmail.com}"
43
44     echo "== schedule it (Cloud Scheduler ? Run job admin API) =="
45     RUN_URI="https://${REGION}-
run.googleapis.com/apis/run.googleapis.com/v1/namespaces/${PROJECT}/jobs/${JOB}:run"
46     gcloud scheduler jobs describe "$SCHED" --location="$REGION" --project="$PROJECT"
>/dev/null 2>&1 && \
47     gcloud scheduler jobs delete "$SCHED" --location="$REGION" --project="$PROJECT"
--quiet || true
48     gcloud scheduler jobs create http "$SCHED" --project="$PROJECT" --location="$REGION" \
49         --schedule="$CRON" --time-zone="$TZ" \
50         --uri="$RUN_URI" --http-method=POST \
51         --oauth-service-account-email="$SA"
52
53     cat <<EOF
54
55     ? Scheduled '$SCHED' ($CRON $TZ) ? Cloud Run job '$JOB' ? launch.sh ? ephemeral GPU VM.
56     IAM the launcher SA needs (grant once if missing):
57         roles/compute.instanceAdmin.v1 (create/delete the GPU VM)
58         roles/iam.serviceAccountUser (attach $SA to the VM)
59         roles/run.invoker + roles/cloudscheduler.jobRunner (Scheduler ? Run)
60     Trigger now to test: gcloud run jobs execute $JOB --region=$REGION
--project=$PROJECT
61     Remove: gcloud scheduler jobs delete $SCHED --location=$REGION; gcloud
run jobs delete $JOB --region=$REGION
62     EOF
63

```

8. user (2026-06-22T09:42:19.450Z)

```

1     # Tiny launcher image for the nightly engine-regression Cloud Run JOB. It is NOT the GPU
workload ?
2     # it only stages the repo + creates the ephemeral GPU VM (which self-deletes). Scale-to-
zero: the job
3     # runs for a few seconds nightly, then exits. Base = google/cloud-sdk (gcloud + git
preinstalled).
4     FROM gcr.io/google.com/cloudsdktool/cloud-sdk:slim

```

```

5     WORKDIR /app
6     # The repo is the build context (run `gcloud builds submit` from the repo root, see
register_scheduler.sh).
7     COPY . /app
8     ENTRYPOINT ["bash", "deploy/gcp/nightly_regression/launch.sh"]
9

```

9. user (2026-06-22T09:42:20.102Z)

```

1     {
2         "_doc": "Nightly E2E engine reel-count manifest (scripts/nightly_reelcount.py). The
configured+checkpointed pipeline runs on each video on a GCP GPU VM; the reel count + quality
metrics are asserted vs eval_baselines/reelcount_baseline.json. ADD A VIDEO = append one row +
re-freeze: `python scripts/nightly_reelcount.py --update-baseline` (commit the updated
baseline). URIs may be gs:// (recommended, co-located with the VM), gdrive://, or local paths.
labels_uri (optional) is a golden-features JSON (gts key) or a start,end CSV ? present =>
quality metrics (tol_f1/f1@0.5/f1@0.25) are scored too. EDIT the example rows below to your
actual short (~5-6 min) OWNED, minors-free clips before enabling.",
3         "segmenter": "wasb_hybrid",
4         "detector_impl": "native",
5         "config_overrides": {
6             "indexing.default_segmenter": "wasb_hybrid",
7             "indexing.detector_impl": "native"
8         },
9         "machine_usd_per_hr": 0.70,
10        "fx_inr_per_usd": 83.0,
11        "videos": [
12            {
13                "name": "GX030094",
14                "uri": "gs://khehsutra-rally-corpus/nightly/videos/GX030094.mp4",
15                "labels_uri": "gs://khehsutra-rally-
corpus/nightly/labels/GX030094_golden_features.json"
16            },
17            {
18                "name": "Badminton_BXH_2",
19                "uri": "gs://khehsutra-rally-corpus/nightly/videos/Badminton_BXH_2.mp4",
20                "labels_uri": "gs://khehsutra-rally-
corpus/nightly/labels/Badminton_BXH_2_golden_features.json"
21            },
22            {
23                "name": "GX010128",
24                "uri": "gs://khehsutra-rally-corpus/nightly/videos/GX010128.mp4",
25                "labels_uri": "gs://khehsutra-rally-
corpus/nightly/labels/GX010128_golden_features.json"
26            }
27        ]
28    }
29

```

10. assistant (2026-06-22T09:42:22.796Z)

Now let me examine the backend to verify integration faithfulness against the actual API contracts.

11. user (2026-06-22T09:42:24.172Z)

```

backend\pipeline\segmenters\__init__.py
backend\pipeline\segmenters\base.py
backend\pipeline\segmenters\_keyhint.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py

```

backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\segmenters\yolo_hybrid.py

12. user (2026-06-22T09:42:25.132Z)

No files found

13. user (2026-06-22T09:42:26.344Z)

backend\eval__init__.py
backend\eval\ablation.py
backend\eval\ablation_pdf.py
backend\eval\annotations.py
backend\eval\batch.py
backend\eval\calibrate_local.py
backend\eval\calibrate_wasb.py
backend\eval\calibration.py
backend\eval\classifier.py
backend\eval\distill_local.py
backend\eval\eval_partial_labels.py
backend\eval\eval_stats.py
backend\eval\experiment.py
backend\eval\export_wasb_segments.py
backend\eval\fusion_audio.py
backend\eval\fusion_compare.py
backend\eval\fusion_golden.py
backend\eval\gemini_refine.py
backend\eval\golden_fixtures.py
backend\eval\golden_real_fixtures.py
backend\eval\gt_loader.py
backend\eval\harness.py
backend\eval\metrics.py
backend\eval\per_user_eval.py
backend\eval\per_user_gate.py
backend\eval\per_user_regression.py
backend\eval\promotion.py
backend\eval\rally_seg_eval.py
backend\eval\rally_seq_proto.py
backend\eval\regression.py
backend\eval\score_calibration.py
backend\eval\segmentation_metrics.py
backend\eval\served_gate_a.py
backend\eval\serve_contrast.py
backend\eval\splits.py
backend\eval\tolerance_metrics.py
backend\eval\training.py
backend\eval>windowing.py

14. user (2026-06-22T09:42:26.804Z)

backend\config__init__.py
backend\config\loader.py
backend\config\models.py
backend\config\presets.py
backend\config\startup.py

15. user (2026-06-22T09:42:27.836Z)

backend\infrastructure\database.py

16. user (2026-06-22T09:42:28.927Z)

backend\billing.py

17. assistant (2026-06-22T09:42:34.682Z)

Let me read the key backend files to verify API contracts.

18. user (2026-06-22T09:42:36.008Z)

```
1      from abc import ABC, abstractmethod
2      from typing import List, Dict, Any, Type, Optional
3      from backend.infrastructure.database import Database
4      from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6      class BasePlaySegmenter(ABC):
7          def __init__(self, db: Database, config: Dict[str, Any]):
8              self.db = db
9              self.config = config
10
11          @property
12          @abstractmethod
13          def name(self) -> str:
14              """Unique identifier for this segmenter."""
15              pass
16
17          @property
18          @abstractmethod
19          def description(self) -> str:
20              """Human-readable explanation of this segmenter."""
21              pass
22
23          @abstractmethod
24          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25              """
26              Processes a video to detect play segments, populates SQLite database,
27              and returns a Result.
28
29              Use isinstance(result, Failure) (or result.is_success()) to distinguish
30              "no segments found" from "something went wrong" (missing key, healthcheck
31              failure, etc.). This is the start of the standardized error/result contract.
32              """
33              pass
34
35      class SegmenterRegistry:
36          def __init__(self):
37              self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39          def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40              """Registers a new segmenter class under its name property."""
41              try:
42                  temp_inst = segmenter_cls(None, {})
43                  name = temp_inst.name
44              except Exception as e:
45                  raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46              self._segmenters[name] = segmenter_cls
47
48          def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49              return self._segmenters.get(name)
50
51          def list_available(self) -> Dict[str, str]:
52              """Returns a dict of registered segmenter names and their descriptions."""
53              res = {}
54              for name, cls in self._segmenters.items():
55                  try:
56                      temp_inst = cls(None, {})
```

```

57         desc = temp_inst.description
58     except Exception:
59         desc = "No description available."
60     res[name] = desc
61     return res
62
63     segmenter_registry = SegmenterRegistry()
64

```

19. user (2026-06-22T09:42:42.931Z)

```

1     from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
2     from backend.pipeline.segmenters.motion import MotionPlaySegmenter
3     from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
4     from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
5     from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
6     from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
7     from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
8
9     __all__ = [
10         "segmenter_registry",
11         "BasePlaySegmenter",
12         "MotionPlaySegmenter",
13         "GeminiPlaySegmenter",
14         "HybridYoloSegmenter",
15         "TrackNetHybridSegmenter",
16         "WasbHybridSegmenter",
17         "FusionSegmenter",
18     ]
19

```

20. user (2026-06-22T09:42:43.728Z)

```

1     """One-command rally-segmentation eval ? score a windowing against the golden labels
with the
2     over-segmentation-sensitive metric set, honestly.
3
4     This is the harness the rally-quality project (`docs/RALLY_QUALITY_RESEARCH.md` $4.5 /
$7)
5     measures every tier against. It closes the loop the over-merge finding exposed: plain
temporal
6     IoU-F1 is blind to merged mega-windows, so we report it alongside segmental F1@k,
the
7     segment-count ratio, and the explicit `merge_split` breakdown
(`segmentation_metrics`).
8
9     For each golden video it: parses the cached WASB/TrackNet trajectory CSV ? builds
candidate
10    windows under a :class:`~backend.eval.windowing.WindowingPreset` (so it tracks the
SERVED
11    operating point and any future windowing the same way) ? scores the windows-as-rallies
against
12    the ground-truth rally intervals. Aggregates per-video held-out F1 with a bootstrap CI,
and can
13    diff two windowings with a paired sign-test + CI (the per-tier `delta`).
14
15    GROUND TRUTH: the golden manifest (`output/human_lovo_manifest.json`) gives each
video's
16    trajectory path + fps + frame_width; the GT rally intervals come from
17    `output/<name>_golden_features.json` (the `gts` key) so this needs no extra label
files.
18
19    Pure offline (CPU): trajectory parsing + windowing + interval scoring; no GPU/Gemini.
20    """
21    from __future__ import annotations

```

```

22
23     import argparse
24     import json
25     import os
26     from typing import Any, Dict, List, Optional, Tuple
27
28     from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
29     from backend.eval.metrics import Interval
30     from backend.eval.windowing import PRESETS, SERVED, WindowingPreset
31
32     DEFAULT_MANIFEST = os.path.join("output", "human_lovo_manifest.json")
33     DEFAULT_FEATURES_DIR = "output"
34     DEFAULT_IOU = 0.5
35
36     #: Long-rally lens threshold (s). A rally is "long" (an eye-catching highlight) when its
duration
37     #: (end ? start) exceeds this. The local detector over-fires, and its false positives
are mostly
38     #: SHORT (motion blips, background-court fragments on multicourt footage); filtering to
long rallies
39     #: strips most of those FPs and reveals the real product quality on the long-rally happy
path. Used
40     #: as the default split point for the stratified report (`--long-tau` overrides it).
DURATION, not
41     #: shot-count, is the filter: golden `shots_count` is unlabeled and the no-AI path
reports it as
42     #: "Unknown", whereas duration is derived from the same start/end labels we already
trust.
43     LONG_RALLY_TAU = 5.0
44
45     #: Multicourt golden clips ? the court-type lookup for the stratified report. The golden
manifest
46     #: (`output/human_lovo_manifest.json`) is gitignored, so this committed set is the
AUDITABLE source
47     #: of the single-vs-multicourt strata, kept in sync with `docs/GOLDEN_VIDEOS.md` (the
clips tagged
48     #: "multicourt"). Multicourt footage carries background games on adjacent courts ? the
dominant
49     #: source of short false positives ? so we report it separately. A manifest entry MAY
carry an
50     #: explicit `court_type` field, which overrides this set (see :func:`court_type_for`).
51     MULTICOURT_CLIPS = frozenset({"mahadevpura_2", "GX010128", "Badminton_BXH_2",
"Boxhill_Doubles"})
52
53
54     def court_type_for(video: Dict[str, Any]) -> str:
55         """`multicourt` or `single` for a golden video. Honors an explicit manifest
56         `court_type` field when present, else falls back to membership in
`MULTICOURT_CLIPS`
57         (keyed by `name`), else `single`."""
58         ct = video.get("court_type")
59         if isinstance(ct, str) and ct:
60             return ct
61         return "multicourt" if video.get("name") in MULTICOURT_CLIPS else "single"
62
63
64     def filter_by_min_duration(ivs: List[Interval], min_duration: float) -> List[Interval]:
65         """The long-rally lens: keep only intervals strictly longer than `min_duration`
seconds.
66
67         `min_duration <= 0` is an EXACT passthrough (the default-OFF semantics ? every
real rally has
68         positive duration, so nothing is dropped). It's a FILTER, not a new metric: applied
identically
69         to predictions AND ground truth before any matching, so the tIoU and R2 paths stay

```

```

consistent."""
70     if min_duration <= 0.0:
71         return list(ivs)
72     return [iv for iv in ivs if (iv[1] - iv[0]) > min_duration]
73
74
75     # ----- #
76     # Pure scoring core (no I/O ? unit-testable)
77     # ----- #
78     def score_intervals(preds: List[Interval], gts: List[Interval],
79                         iou: float = DEFAULT_IOU,
80                         tau_start: Optional[float] = None,
81                         tau_end: Optional[float] = None,
82                         min_duration: float = 0.0) -> Dict[str, Any]:
83         """The full per-video metric row for predicted vs ground-truth rally intervals.
84
85         Pairs temporal-IoU P/R/F1 + boundary error (`metrics.score`) with the
86         over-segmentation-sensitive set (F1@k, segment-count ratio, merge/split breakdown)
87         so a
88         merged mega-window is *visible* even when IoU-F1 isn't moved.
89
90         Also carries the **R2** task-faithful block (`tolerance_metrics`, the headline
91         going forward;
92         tIoU here is the SECONDARY trend-line per the augment-then-ablation-gate decision):
93         strict-1:1
94         tolerance-match `tol_f1/tol_precision/tol_recall` (over-production costs
95         precision) + the
96         start/end boundary-error medians + the over-seg guard counts. See
97         docs/R2_EVAL_METRIC_DESIGN.md.
98
99         ``min_duration`` > 0 applies the **long-rally lens**
100         (:func:`filter_by_min_duration`): BOTH preds
101         and GTs are filtered to rallies longer than that many seconds BEFORE any matching,
102         so the whole
103         row (tIoU set AND R2 block) scores only long rallies. Default 0.0 = OFF (exact
104         passthrough).
105         """
106         from backend.eval import tolerance_metrics as tm
107         preds = filter_by_min_duration(preds, min_duration)
108         gts = filter_by_min_duration(gts, min_duration)
109         ts = tm.TAU_START if tau_start is None else tau_start
110         te = tm.TAU_END if tau_end is None else tau_end
111         sc = metrics.score(preds, gts, iou_threshold=iou)
112         f1k = segmentation_metrics.f1_at_overlaps(preds, gts)
113         ms = segmentation_metrics.merge_split_report(preds, gts)
114         tol_matches, tol_p, tol_r, tol_f1 = tm.match_1to1(preds, gts, ts, te)
115         be = tm.boundary_error_report(preds, gts) # unconditional best-overlap (the
116         R4-aiming diagnostic)
117         osr = tm.over_seg_report(preds, gts)
118         return {
119             "f1": sc.f1, "precision": sc.precision, "recall": sc.recall, "mean_iou":
120             sc.mean_iou,
121             "mean_start_error": sc.mean_start_error, "mean_end_error": sc.mean_end_error,
122             "f1@0.1": f1k[0.1], "f1@0.25": f1k[0.25], "f1@0.5": f1k[0.5],
123             "segment_count_ratio": segmentation_metrics.segment_count_ratio(preds, gts),
124             "merge_rate": ms["merge_rate"], "split_rate": ms["split_rate"],
125             "merges": ms["merges"], "splits": ms["splits"], "missed": ms["missed"],
126             "spurious": ms["spurious"], "num_pred": len(preds), "num_gt": len(gts),
127             # --- R2 (tolerance-match) ? the task-faithful headline block ---
128             "tau_start": ts, "tau_end": te,
129             "tol_f1": tol_f1, "tol_precision": tol_p, "tol_recall": tol_r,
130             "tol_matched": float(len(tol_matches)),
131             "dstart_abs_median": be["dstart"]["abs_median"], "dend_abs_median":
132             be["dend"]["abs_median"],
133             "split_count": osr["split_count"], "merge_count": osr["merge_count"],

```

```

123     }
124
125
126     _AGG_KEYS = ("f1", "f1@0.25", "f1@0.5", "merge_rate", "split_rate",
127 "segment_count_ratio",
128 "precision", "recall",
129 "tol_f1", "tol_precision", "tol_recall", "dstart_abs_median",
130 "dend_abs_median")
131
132     def aggregate(rows: List[Dict[str, Any]]) -> Dict[str, Any]:
133         """Aggregate per-video rows: mean (+ bootstrap 95% CI) of the headline metrics
134 across videos.
135
136 Per-video means (not pooled) keep videos the unit of evidence, matching the
137 harness's
138 leave-one-video-out discipline (windows within a video are correlated)."""
139 out: Dict[str, Any] = {"n": len(rows)}
140 for k in _AGG_KEYS:
141     vals = [r[k] for r in rows]
142     out[k] = eval_stats.mean_with_ci(vals) if vals else None
143 return out
144
145 # ----- #
146 # Golden corpus I/O
147 # ----- #
148 def load_golden(manifest_path: str = DEFAULT_MANIFEST,
149 features_dir: str = DEFAULT_FEATURES_DIR) -> List[Dict[str, Any]]:
150     """Load each golden video's trajectory path + fps + frame_width (manifest) and GT
151 rally
152 intervals (`<name>_golden_features.json` `gts`). Videos with no GTs are kept but
153 flagged."""
154 with open(manifest_path) as fh:

```

21. user (2026-06-22T09:42:44.433Z)

```

1     """Configuration loader and saver.
2
3     Provides a single place to load validated AppConfig with built-in defaults
4     merged with optional file overrides (config.json).
5
6     This replaces scattered json.load + .get() patterns throughout the codebase.
7     """
8
9     from __future__ import annotations
10
11     import json
12     import logging
13     import os
14     from pathlib import Path
15     from typing import Any
16
17     from pydantic import BaseModel
18
19     from backend.utils.app_home import default_config_path
20
21     from .models import AppConfig
22     from .presets import (
23         COMPUTE_TARGET_FOR_PROFILE,
24         PROFILE_PRESETS,
25         is_known_profile,
26         preset_for,
27         suggest_profile,
28     )

```



```

29
30     logger = logging.getLogger(__name__)
31
32     # Config path resolution now lives in ``backend.utils.app_home.default_config_path()`` ?
it honours
33     # ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a) and equals ``Path("config.json")`` (CWD-
relative) when
34     # the env is unset, exactly the pre-P0a value. (The old module-level
``DEFAULT_CONFIG_PATH`` constant
35     # was dropped: it had no external consumers and a frozen-at-import snapshot would
silently diverge
36     # from a later RALLY_APP_HOME.)
37
38
39     def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
40         """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge,
scalars
41         replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
42         overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-
profile
43         path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""
44         out = dict(base)
45         for key, val in override.items():
46             if isinstance(val, dict) and isinstance(out.get(key), dict):
47                 out[key] = _deep_merge(out[key], val)
48             else:
49                 out[key] = val
50         return out
51
52
53     def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
54         """The EXPLICITLY chosen deployment profile, env winning over config.json. ``""`` if
none
55         is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so
a bare
56         environment never silently selects a profile (and never silently spends on cloud).
57
58         An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the
file's
59         profile (rather than suppressing it) ? so an env typo can't leave the recorded
profile
60         asserting a preset that was never applied. A bad value in config.json is left to
surface as
61         a hard, typed-Literal error downstream (config-file correctness)."""
62         env_profile = os.environ.get("DEPLOYMENT_PROFILE")
63         if env_profile and env_profile.strip():
64             ep = env_profile.strip()
65             if is_known_profile(ep):
66                 return ep
67             logger.warning(
68                 "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring
it "
69                 "and falling back to config.json deployment.profile.", ep, ",
".join(PROFILE_PRESETS),
70             )
71             # fall through to the file's profile (env typo must not suppress a valid file
choice)
72         dep = file_data.get("deployment")
73         if isinstance(dep, dict):
74             prof = dep.get("profile")
75             if isinstance(prof, str) and prof.strip():
76                 return prof.strip()
77         return ""
78
79

```

```

80     def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
81     "") -> list[str]:
82         """Dotted paths in `data` that the typed config will not honor.
83
84         Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
85         kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
86         typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
87         user's intent is lost ? collect every such key so load_config can warn.
88         """
89         unknown: list[str] = []
90         fields = model_cls.model_fields
91         for key, val in data.items():
92             if key not in fields:
93                 unknown.append(prefix + key)
94                 continue
95             ann = fields[key].annotation
96             if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
97             BaseModel):
98                 unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}.")
99         return unknown
100
101     def _get_builtin_defaults() -> dict[str, Any]:
102         """Return a plain dict of the current built-in defaults.
103
104         The AppConfig model is the single source of truth for defaults.
105         """
106         return AppConfig().model_dump(mode="json")
107
108     def load_config(path: str | Path | None = None) -> AppConfig:
109         """Load configuration with the following precedence:
110
111         1. Built-in defaults defined in the Pydantic models.
112         2. Values from the JSON file (if present and readable).
113         3. (Future) environment / CLI overrides.
114
115         Missing file or unreadable file ? returns a fully defaulted AppConfig.
116         Unknown keys in the file are tolerated (extra="allow" on the model).
117         """
118         cfg_path = Path(path) if path is not None else default_config_path()
119
120         file_data: dict[str, Any] = {}
121         if cfg_path.exists():
122             try:
123                 with open(cfg_path, "r", encoding="utf-8") as f:
124                     file_data = json.load(f)
125             except Exception as exc:
126                 # Non-fatal: continue with defaults + whatever we could parse.
127                 # In production we might log here.
128                 logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
129                 data.")
130
131         # A config.json that is valid JSON but not an OBJECT (e.g. `[ ]`, `42`, `"x"`,
132         # `null`) would
133         # otherwise crash startup: a truthy non-mapping hits `.items()` in
134         # _unknown_key_paths, and any
135         # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults +
136         # warn, per the
137         # loader's "bad input is non-fatal" contract.
138         if not isinstance(file_data, dict):
139             logger.warning("%s is not a JSON object (got %s); ignoring it and using
140             defaults.",
141                             cfg_path, type(file_data).__name__)
142             file_data = {}

```

```

138
139     if file_data:
140         unknown = _unknown_key_paths(file_data, AppConfig)
141         if unknown:
142             logger.warning(
143                 "[CONFIG WARNING] %s contains keys the typed config does not "
144                 "recognize (typo'd or removed knobs ? top-level extras are kept "
145                 "but unread; unknown section keys are silently dropped): %s",
146                 cfg_path, ", ".join(sorted(unknown)),
147             )
148
149     # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied
at
150     # consumption sites downstream). A *deployment profile* is opt-in ? it is applied
ONLY when
151     # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile).
With no
152     # profile the merge is byte-identical to the pre-M1 loader, so the default path is
unchanged.
153     defaults = _get_builtin_defaults()
154     profile = _resolve_explicit_profile(file_data)
155
156     if profile and not is_known_profile(profile):
157         # Only an unrecognised value from config.json reaches here (env typos already
fell back
158         # in _resolve_explicit_profile). Warn + apply no preset; the bad value also hits
the
159         # typed Literal at model_validate below, a hard clear error ? loud, never
silent.
160         logger.warning(
161             "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no
preset.",
162             profile, ", ".join(PROFILE_PRESETS),
163         )
164         profile = ""
165
166     if profile:
167         merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
168         # Record the profile actually applied ? the env may have chosen it over
config.json,
169         # so re-stamp it onto the merged deployment section to keep the record honest.
170         merged.setdefault("deployment", {})
171         if isinstance(merged["deployment"], dict):
172             merged["deployment"]["profile"] = profile
173             ct = merged["deployment"].get("compute_target")
174             canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
175             if not ct and canonical:
176                 # An active profile with no explicit compute_target (e.g. config.json
blanked it
177                 # to "") gets its canonical target back ? the record never silently lies
about an
178                 # active profile's compute_target (it stays meaningful for the M2+
ComputeBackend).
179                 merged["deployment"]["compute_target"] = canonical
180             elif ct and canonical and ct != canonical:
181                 logger.warning(
182                     "[CONFIG] deployment profile %r normally uses compute_target=%r but
"
183                     "config.json overrides it to %r ? honoring the explicit override.",
184                     profile, canonical, ct,
185                 )
186                 logger.info("[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
187                             profile, merged["deployment"].get("compute_target"))
188     else:
189         # No profile ? the original shallow merge (Pydantic re-defaults nested

```

```

sections).
190         merged = {**defaults, **file_data}
191         suggestion = suggest_profile() # cheap, env-only (no torch); SUGGESTS only
(D15)
192         if suggestion:
193             logger.debug(
194                 "[CONFIG] no deployment.profile set (manual knobs). Environment suggests
%r ? "
195                 "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect
never "
196                 "auto-applies ? D15).", suggestion,
197             )
198
199         return AppConfig.model_validate(merged)
200
201
202     def save_default_config(path: str | Path | None = None) -> Path:
203         """Write a fresh default config.json.
204
205         Used by scripts/doctor.py diagnostics and the --setup CLI path.
206         Overwrites if the file exists.
207         """
208         cfg_path = Path(path) if path is not None else default_config_path()
209         cfg = AppConfig()
210         cfg_path.parent.mkdir(parents=True, exist_ok=True)
211         cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
212         return cfg_path
213
214
215     # The shipped batteries-included LIGHT-tier default (PACKAGING_PLAN.md P2b / gate 08).
216     # Deliberately MINIMAL: the `truly-local` PROFILE preset (presets.py) supplies
skip_ai_handoff,
217     # local storage and the compute_target, so this stays the handful of knobs a fresh, non-
technical
218     # user gets ? a torch-free, offline, zero-setup experience:
219     # * default_segmenter "motion" ? pure-OpenCV, no torch / GPU / weights / API key
(LIGHT tier);
220     # * profile "truly-local" ? no Gemini handoff, local storage ? never surprise-
bills on
221     # Gemini nor returns 0 rallies on WASB-without-a-key
(08 intent);
222     # * use_gpu false ? honest for a LIGHT (no-GPU) box (motion ignores it
either way).
223     # The first-run wizard (P2c) upgrades this to Gemini once the user supplies a key.
224     LIGHT_DEFAULT_CONFIG: dict[str, Any] = {
225         "sport": "badminton",
226         "deployment": {"profile": "truly-local"},
227         "indexing": {"default_segmenter": "motion", "use_gpu": False},
228     }
229
230
231     def write_light_default_config(
232         path: str | Path | None = None, *, overwrite: bool = False
233     ) -> tuple[Path, bool]:
234         """Seed the batteries-included LIGHT default config (`LIGHT_DEFAULT_CONFIG`).
235
236         Returns `(path, written)`. Unlike :func:`save_default_config` (which dumps a FULL
AppConfig
237         and overwrites), this writes the minimal profile-driven LIGHT bundle and ? by
default ? **never
238         overwrites an existing file**, so a user's edits or the first-run wizard's choices
are preserved
239         (idempotent first-run seeding). Pass ``overwrite=True`` to force. Creates parent
dirs.
240         """

```

```

241         cfg_path = Path(path) if path is not None else default_config_path()
242         if cfg_path.exists() and not overwrite:
243             return cfg_path, False
244         cfg_path.parent.mkdir(parents=True, exist_ok=True)
245         cfg_path.write_text(json.dumps(LIGHT_DEFAULT_CONFIG, indent=2) + "\n",
encoding="utf-8")
246         return cfg_path, True
247
248
249     # Backwards-compatible alias used by some older call sites during transition.
250     def get_default_config() -> dict[str, Any]:
251         """Return defaults as a plain dict (for transitional use only)."""
252         return _get_builtin_defaults()
253

```

22. user (2026-06-22T09:42:44.748Z)

```

1     import os
2     import shutil
3     import sqlite3
4     import json
5     import logging
6     from contextlib import contextmanager
7     from typing import List, Dict, Any, Optional, Tuple
8
9     from backend import billing # pure cost math (no backend imports ? no cycle)
10
11     logger = logging.getLogger(__name__)
12
13     # The highest schema version this binary knows how to migrate TO (the top of the
migration ladder
14     # in `_init_db`). PACKAGING_PLAN.md P2d uses it for the upgrade-safety contract on a
downloadable,
15     # auto-updating app: refuse to open a DB from a NEWER app (downgrade guard) and back the
DB up before
16     # an upgrade migration runs.
`tests/test_database.py::test_schema_version_matches_ladder` pins it to
17     # the ladder so adding a `_migrate_to_vN` without bumping this fails CI.
18     SCHEMA_VERSION = 7
19
20
21     class SchemaTooNewError(RuntimeError):
22         """The on-disk DB was created by a newer app version than this binary understands
(P2d)."""
23
24     class Database:
25         def __init__(self, db_path: str):
26             self.db_path = db_path
27             self._init_db()
28
29         @contextmanager
30         def _connect(self):
31             """Transaction scope that also CLOSES the connection.
32
33             ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
never closes, so every call leaked its connection to GC (lingering file
handles on the WAL sidecars under Windows). Internal methods use this.
36             """
37             conn = self._get_connection()
38             try:
39                 with conn:
40                     yield conn
41             finally:
42                 conn.close()
43

```

```

44     def _get_connection(self):
45         # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
46         # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
47         conn = sqlite3.connect(self.db_path, timeout=30.0)
48         conn.row_factory = sqlite3.Row
49         # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
50         conn.execute("PRAGMA foreign_keys = ON")
51         # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
52         conn.execute("PRAGMA busy_timeout = 30000")
53         try:
54             conn.execute("PRAGMA journal_mode = WAL")
55         except sqlite3.OperationalError:
56             pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
57         return conn
58
59     def _execute_write(self, sql: str, params: Tuple[Any, ...], error_msg: str) -> bool:
60         """Run ONE bool-returning single-statement write inside a connection scope.
61
62         Captures the connect/commit/except-logger.error/return-False boilerplate shared
63         by
64         the bool-returning single-statement writers. ``error_msg`` is the caller's exact
65         per-method message (already formatted with its own ids) ? logged verbatim on a
66         swallowed write fault so each writer keeps its specific log text. Returns True
67         on
68         success, False (after logging ``error_msg``) on any exception."""
69         try:
70             with self._connect() as conn:
71                 conn.cursor().execute(sql, params)
72                 conn.commit()
73             return True
74         except Exception as e:
75             logger.error(f"{error_msg}: {e}")
76             return False
77
78     @staticmethod
79     def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
80         """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runably.
81
82         A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn``
83         transaction, so an interrupted v3/v4 block can leave the column added while
84         ``user_version`` stays un-bumped ? and the next open would re-run the same
85         ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
86         future ``Database(...)`` construction (``_init_db`` runs in ``__init__``).
87         Swallow
88         ONLY that specific error so the ladder stays crash-safe, matching the re-
89         runnable
90
91         ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
92         """
93         try:
94             cursor.execute(ddl)
95         except sqlite3.OperationalError as exc:
96             if "duplicate column name" not in str(exc).lower():
97                 raise
98
99     def _init_db(self):
100         """Initializes tables for videos, play_segments, and events."""
101         with self._connect() as conn:
102             cursor = conn.cursor()
103
104             self._create_base_tables(cursor)

```

```

1      """Per-job cost ledger ? pure cost math (COMPUTE_DECOUPLED_SERVING M8, decision D8).
2
3      ``cost_usd(job) = ?_resource usage(job, resource) × rate(resource, pricebook_version)``
4      (the formula from the archived 05-COST-ATTRIBUTION design). Cost is computed + stored in
5      **USD**
6      (matches GCP billing = one source of truth) and **displayed in INR** at a configurable
7      FX rate
8      (the Bangalore market). The pricebook is a versioned DB table
9      (``backend/infrastructure/database.py``);
10     this module is the pure arithmetic over a resolved ``{resource: unit_price_usd}`` rates
11     dict, so it
12     is unit-tested with zero IO and the DB layer owns the version/gpu_type lookup.
13
14     **Default-OFF:** with the seeded ``placeholder-v0`` pricebook (every rate ``0.0``) every
15     cost is
16     ``0.0`` ? nothing user-facing changes until the owner inserts a real, calibrated
17     pricebook version.
18     """
19
20     from __future__ import annotations
21
22     from typing import Dict, Mapping, Tuple
23
24     # Metered resource keys ? shared by the usage dict, the pricebook rows, and the
25     breakdown.
26     GPU_SECONDS = "gpu_seconds"      # GPU-active seconds (the big one) ? rate is per-second,
27     per gpu_type
28     WALL_SECONDS = "wall_seconds"    # total wall time (separate from GPU so idle/sync isn't
29     billed as GPU)
30     EGRESS_GB = "egress_gb"          # reel/byte egress in GB
31     GEMINI_USD = "gemini_usd"        # provider-reported cost, ALREADY in USD (a passthrough,
32     not rate-x)
33
34     # Resources whose cost = amount × unit_price (GEMINI_USD is excluded ? it is already a
35     USD amount).
36     _RATE_MULTIPLIED = (GPU_SECONDS, WALL_SECONDS, EGRESS_GB)
37
38     def compute_cost_usd(usage: Mapping[str, float],
39                          rates: Mapping[str, float]) -> Tuple[float, Dict[str, float]]:
40         """Return ``(cost_usd, breakdown)`` for one job.
41
42         ``usage`` maps a resource key to its measured amount (seconds / GB); ``rates`` maps
43         a resource
44         key to its USD unit price (already resolved for this pricebook version + gpu_type by
45         the DB
46         layer). ``GEMINI_USD`` in ``usage`` is added verbatim (it is already a USD cost, not
47         multiplied).
48         Missing amounts/rates default to ``0.0`` ? with the placeholder pricebook the total
49         is ``0.0``.
50         """
51         breakdown: Dict[str, float] = {}
52         total = 0.0
53         for res in _RATE_MULTIPLIED:
54             amount = float(usage.get(res, 0.0) or 0.0)
55             rate = float(rates.get(res, 0.0) or 0.0)
56             cost = round(amount * rate, 6)      # round the LINE, then sum rounded lines so
57             the
58             if amount or rate:                  # breakdown reconciles to the total exactly
59             (no drift),
60             breakdown[res] = cost              # and record the line even at $0 for
61             transparency
62             total += cost
63         gemini = round(float(usage.get(GEMINI_USD, 0.0) or 0.0), 6)
64         if gemini:

```

```

48         breakdown[GEMINI_USD] = gemini
49     total += gemini
50     return round(total, 6), breakdown
51
52
53     def apply_margin(cost_usd: float, margin: float) -> float:
54         """Resale price = `cost_usd * (1 + margin)``. ``margin`` is clamped ? 0 (never
below cost)."""
55         return round(cost_usd * (1.0 + max(0.0, float(margin))), 6)
56
57
58     def to_inr(cost_usd: float, fx_inr_per_usd: float) -> float:
59         """Display conversion: USD ? INR at the configured FX rate (2 dp, the display
currency)."""
60         return round(float(cost_usd) * float(fx_inr_per_usd), 2)
61

```

24. assistant (2026-06-22T09:42:48.090Z)

Now let me search for backend.storage.fetch to verify the API.

25. user (2026-06-22T09:42:49.405Z)

(Bash completed with no output)

26. user (2026-06-22T09:42:50.141Z)

```

Found 9 files
backend\storage\s3.py
backend\storage\local.py
backend\storage\gdrive.py
backend\storage\gcs.py
backend\storage\base.py
backend\storage\__init__.py
backend\annotations\file_provider.py
backend\annotations\base.py
backend\annotations\automated_provider.py

```

27. user (2026-06-22T09:42:52.560Z)

```

1     """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3     URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4     lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5     ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6     today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7     Google Drive (Google Drive for Desktop) as plain files.
8
9     Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10    ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11    passthrough. See ``docs/DECOUPLED_COMPUTE``.
12    """
13    from __future__ import annotations
14
15    import os
16    import tempfile
17    from typing import Any, Dict, Iterator, Optional
18
19    from backend.storage.base import StorageBackend
20    from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri

```



```

21
22     __all__ = [
23         "StorageBackend", "StorageRef", "StorageStat", "parse_uri", "resolve_input_uri",
24         "get_storage", "fetch", "put", "list_uris",
25         "acquire_local_input", "input_is_local", "resolve_input_ref",
26     ]
27
28
29     def _storage_dict(storage_config: Any) -> Dict[str, Any]:
30         """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
31         if storage_config is None:
32             return {}
33         if hasattr(storage_config, "model_dump"):
34             return storage_config.model_dump() # type: ignore[no-any-return]
35         return dict(storage_config)
36
37
38     def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
39         """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
40         config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
41         callers at the API/CLI boundary catch that and return a clean client error (never a
500).
42
43         Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
44         ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
45         sc = _storage_dict(storage_config)
46         uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
47                               input_root=sc.get("input_root", ""))
48         return parse_uri(uri)
49
50
51     def input_is_local(raw: str, storage_config: Any = None) -> bool:
52         """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
53         plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
54         that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
55         ``resolve_input_ref``)."""
56         return resolve_input_ref(raw, storage_config).backend == "local"
57
58
59     def acquire_local_input(raw: str, storage_config: Any = None, *,
60                            scratch_parent: "str | None" = None) -> str:
61         """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
62
63         Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
64         byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
65         non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
66         path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
67         regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
68         worker's ``cmd_infer`` already ships."""
69         sc = _storage_dict(storage_config)
70         ref = resolve_input_ref(raw, sc)
71         if ref.backend == "local":
72             return ref.key # identity ? no temp, no copy, byte-identical to passing the

```

```

path directly
73     scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
74     return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
75
76
77     def _backend_class(name: str):
78         """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
79         if name == "local":
80             from backend.storage.local import LocalStorage
81             return LocalStorage
82         if name == "s3":
83             from backend.storage.s3 import S3Storage
84             return S3Storage
85         if name == "gcs":
86             from backend.storage.gcs import GCSStorage
87             return GCSStorage
88         if name == "gdrive":
89             from backend.storage.gdrive import GDriveStorage
90             return GDriveStorage
91         raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")
92
93
94     def get_storage(backend: Optional[str] = None, *,
95                     config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
96         """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
97         the implementation; its per-backend settings come from ``config[backend]`` merged
with
98         ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
99         cfg = config or {}
100         name = backend or cfg.get("backend", "local")
101         settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
102         settings.update(overrides)
103         return _backend_class(name)(**settings)
104
105
106     def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
107              config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
108         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
109         (no copy); for s3/gcs it downloads to ``dst``."""
110         ref = parse_uri(uri)
111         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
112
113
114     def put(local_path: "str | os.PathLike[str]", uri: str, *,
115            config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
116         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
117         ref = parse_uri(uri)
118         get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
119         return uri
120
121
122     def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
Iterator[str]:
123         """Yield the URI of every object under ``prefix_uri``."""
124         ref = parse_uri(prefix_uri)
125         for r in get_storage(ref.backend, config=config).list(ref):
126             yield r.uri
127

```

28. user (2026-06-22T09:42:53.223Z)

```
1      """The ``StorageBackend`` ABC ? one interface for local + object-store + Drive.
2
3      Mirrors the codebase's ABC+Registry idiom (cf. ``SegmenterRegistry``). The bucket
backends
4      live in sibling modules and lazy-import their cloud SDK so ``import backend.storage``
never
5      pulls ``boto3`` / ``google-cloud-storage`` (``gdrive`` is pure local-FS over a mounted
Drive ?
6      no SDK). See ``PLATFORM_ARCHITECTURE.md`` §3.1 and ``docs/DECOUPLED_COMPUTE/``.
7      """
8      from __future__ import annotations
9
10     import os
11     from abc import ABC, abstractmethod
12     from typing import Iterator
13
14     from backend.storage.ref import StorageRef, StorageStat
15
16
17     class StorageBackend(ABC):
18         """Backend-agnostic object storage. ``fetch_to_local`` + ``put`` are the two the
19         worker dispatcher needs; the rest support listing/checkpointing/serving."""
20
21         @abstractmethod
22         def fetch_to_local(self, ref: StorageRef, dst: "str | os.PathLike[str] | None" =
None,
23                             *, overwrite: bool = False) -> str:
24             """Make the object available as a local file; return its local path. For the
25             ``local`` backend this is an identity passthrough (the object is already
local)."""
26
27         @abstractmethod
28         def put(self, local_path: "str | os.PathLike[str]", ref: StorageRef, *,
29                 content_type: "str | None" = None) -> StorageRef:
30             """Upload/copy a local file to ``ref``; return ``ref``."""
31
32         @abstractmethod
33         def exists(self, ref: StorageRef) -> bool: ...
34
35         @abstractmethod
36         def stat(self, ref: StorageRef) -> StorageStat: ...
37
38         @abstractmethod
39         def list(self, prefix: StorageRef) -> Iterator[StorageRef]:
40             """Yield a ``StorageRef`` per object under ``prefix``."""
41
42         @abstractmethod
43         def delete(self, ref: StorageRef) -> None: ...
44
45         def signed_url(self, ref: StorageRef, *, ttl: int = 3600, method: str = "GET") ->
str:
46             """A time-limited URL handing the bytes to a browser/LLM. Not every backend can
47             sign (``local`` cannot; Drive only emulates) ? callers must treat
48             ``NotImplementedError`` as expected and fall back to ``fetch_to_local``."""
49             raise NotImplementedError(f"{type(self).__name__} cannot produce signed URLs")
50
```

29. user (2026-06-22T09:42:54.198Z)

```
1      """Google Cloud Storage backend ? first-class object buckets.
2
3      Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
```

```

4      install with the ``storage-gcs`` extra. Credentials come from Application Default
Credentials
5      (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
identity) ?
6      never from config. A ``client`` may be injected for tests.
7      """
8      from __future__ import annotations
9
10     from datetime import timedelta
11     from typing import Any, Iterator, Optional
12
13     from backend.storage.base import StorageBackend
14     from backend.storage.ref import StorageRef, StorageStat
15
16
17     class GCSStorage(StorageBackend):
18         def __init__(self, *, client: Any = None, project: Optional[str] = None,
19                     signed_url_ttl: int = 3600, **_ignored) -> None:
20             self._ttl = signed_url_ttl
21             if client is not None:
22                 self._client = client
23             return
24             from google.cloud import storage # type: ignore # lazy: only when a gcs:// URI
is used
25                 self._client = storage.Client(project=project)
26
27         def _blob(self, ref: StorageRef):
28             return self._client.bucket(ref.bucket).blob(ref.key)
29
30         def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
31             if dst is None:
32                 raise ValueError("GCSStorage.fetch_to_local requires a dst path")
33             import os
34             dst = str(dst)
35             if os.path.exists(dst) and not overwrite:
36                 return dst
37             os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
38             self._blob(ref).download_to_filename(dst)
39             return dst
40
41         def put(self, local_path, ref, *, content_type=None) -> StorageRef:
42             self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
43             return ref
44
45         def exists(self, ref) -> bool:
46             return bool(self._blob(ref).exists())
47
48         def stat(self, ref) -> StorageStat:
49             b = self._blob(ref)
50             b.reload()
51             return StorageStat(size=int(b.size or 0), modified=b.updated, etag=b.etag,
52                               content_type=b.content_type)
53
54         def list(self, prefix) -> Iterator[StorageRef]:
55             for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
56                 yield StorageRef("gcs", prefix.bucket, blob.name,
f"gcs://{prefix.bucket}/{blob.name}")
57
58         def delete(self, ref) -> None:
59             self._blob(ref).delete()
60
61         def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
62             return self._blob(ref).generate_signed_url(
63                 version="v4", expiration=timedelta(seconds=ttl or self._ttl), method=method)
64

```

30. user (2026-06-22T09:42:54.934Z)

```
1      """Lightweight Result type for standardizing success vs. failure across the pipeline.
2
3      This is the start of the Error/Result Contract standardization (low-regret item #3).
4
5      The goal is to make it impossible to confuse "no data / no rallies" with "something
6      actually failed"
7      (e.g. missing API key, WSL healthcheck failure, provider error, etc.).
8
9      We use simple dataclasses so callers can use isinstance checks or structural pattern
10     matching.
11
12     This is intentionally small and incremental ? we start by applying it to the segmenter
13     and provider layers (the biggest sources of silent [] returns today).
14
15     The types are generic where it makes sense. Callers can still access .value on Success
16     or .message on Failure.
17
18     This module has no dependencies on the rest of the app.
19     """
20
21     from __future__ import annotations
22
23     from dataclasses import dataclass, field
24     from typing import Generic, TypeVar, List, Optional, Any, Tuple, Union
25
26     T = TypeVar("T")
27
28     @dataclass
29     class Success(Generic[T]):
30         """Represents a successful outcome with an optional list of non-fatal warnings."""
31
32         value: T
33         warnings: List[str] = field(default_factory=list)
34
35         def is_success(self) -> bool:
36             return True
37
38         def unwrap(self) -> T:
39             """Return the value. Only safe to call if you already know it's a Success."""
40             return self.value
41
42     @dataclass
43     class Failure:
44         """Represents a failure with a human-readable message and optional details."""
45
46         message: str
47         details: Optional[dict[str, Any]] = None
48         is_recoverable: bool = False # e.g. missing API key (recoverable) vs internal crash
49
50         def is_success(self) -> bool:
51             return False
52
53         def unwrap(self) -> Any:
54             """Raise to make misuse obvious in code that assumes success."""
55             raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
56
57     # Convenient alias for use in type hints: Result[list[dict], Failure]
58     # Result[ValueType] means Success[ValueType] | Failure
59     # (the Failure "error" type is fixed for this contract)
60     Result = Union[Success[T], Failure]
61
```

```

62
63     def unwrap_or_failure(result: Any) -> Tuple[Optional[Any], Optional[Failure]]:
64         """Split a segmenter/provider return into ``(value, failure)`` so a caller branches
once.
65
66         - ``Failure``          -> ``(None, failure)``
67         - ``Success(value=v)`` -> ``(v, None)``
68         - a bare value (legacy code that predates this contract and still returns a raw
list)
69         -> ``(value, None)``
70
71         This is the same isinstance/``.value`` unwrap that ``backend/main.py`` does inline;
callers
72         that forget it (e.g. ``len(process_video(...))``) crash with ``TypeError: ... has no
len()``
73         on a ``Success`` dataclass. ``result`` is typed ``Any`` because the contract is
heterogeneous
74         (Success | Failure | legacy bare value)."""
75         if isinstance(result, Failure):
76             return None, result
77         value = result.value if hasattr(result, "value") else result
78         return value, None
79

```

31. assistant (2026-06-22T09:42:57.339Z)

Now let me check the config models to see what fields are accessible and verify dotted override targets:

32. user (2026-06-22T09:42:58.675Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     The models support graceful migration from old config.json files via extra="allow".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33     class TrackNetConfig(BaseModel):

```

```

34     wsl_distro: str = "Ubuntu"
35     repo_dir: str = "~/models/TrackNetV4"
36     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37     conda_env: str = "TrackNetV4"
38     weights_path: str = ""
39     queue_length: int = 5
40     stage_in_wsl: bool = True
41     wsl_stage_dir: str = "~/clips"
42     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43     velocity_threshold: float = 0.02
44     # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45     # decoded frame cache) are deleted automatically ? they are pure intermediates,
46     # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47     # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48     keep_frames: bool = False
49     # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50     # 0 = disabled.
51     min_free_gb: float = 0.0
52
53
54     class WasbIndexConfig(BaseModel):
55         """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58         these knobs actually take effect ? previously this whole block was silently dropped
59         because nested config sections defaulted to extra='ignore'.
60         """
61         wsl_distro: str = "Ubuntu"
62         repo_dir: str = "~/models/WASB-SBDT"
63         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64         conda_env: str = "wasb"
65         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66         sport: str = "badminton"
67         wsl_stage_dir: str = "~/clips"
68         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69         velocity_threshold: float = 0.02
70         # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
71         ---
72         # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
73         # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
74         # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
75         device: str = "cuda" # auto | cuda | mps | cpu
76         python_bin: str = "python" # the `wasb` conda env python (invoked by path, no
`conda activate`)
77         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
78         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
79         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
80         keep_frames: bool = False
81         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
82         min_free_gb: float = 0.0
83         # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
84         # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
85         # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
86         # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
87         # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).

```

```

87         # Tri-state:
88         #     None          = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
89         #                               behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
90         #     True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
91         stream_video: Optional[bool] = None
92
93
94     class FusionConfig(BaseModel):
95         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
96
97         Every default reproduces control behaviour: the fusion segmenter persists the
98         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
99         true ? the person-cue features, but it does NOT gate the served handoff unless a
100        threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
101        is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
102        supplied ? off-court persons (spectators / adjacent courts) are excluded.
103        """
104        # Which trajectory substrate seeds the windows (shuttle stays primary).
105        substrate: Literal["wasb", "tracknet"] = "wasb"
106        # Windowing velocity threshold for the fusion family (the engine reads
107        # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
108        # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
109        velocity_threshold: float = 0.02
110        # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
111        # enabled ? so the default path never imports torch / touches the GPU.
112        cue_flags: dict[str, bool] = Field(default_factory=dict)
113        # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
114        # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
115        min_crossings: int = Field(default=0, ge=0)
116        # Served Gate B: when the person cue is on, drop windows with median in-court
players
117        # < this. 0 = OFF.
118        min_players: int = Field(default=0, ge=0)
119        # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
120        court_polygon: Optional[list[list[float]]] = None
121        # Net line for the person two-sided-motion split (person features only ? the shuttle
122        # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
123        net_axis: Literal["x", "y"] = "y"
124        net_position: float = Field(default=0.5, ge=0.0, le=1.0)
125
126
127     class IndexingConfig(BaseModel):
128         default_segmenter: str = "yolo_hybrid"
129         use_gpu: bool = True
130         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
131         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
132         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
133         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
134         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
135         detector_impl: str = "auto"
136         motion_threshold: float = 0.08
137         min_segment_duration: float = 3.0
138         dead_time_merge_gap: float = 2.0
139         chunk_duration: float = 90.0
140         chunk_overlap: float = 10.0

```



```

141     detector_model: str = "fasterrcnn_resnet50_fpn"
142     detector_score_threshold: float = 0.5
143     yolo_velocity_threshold: float = 0.15
144     yolo_frame_skip: int = 3
145     yolo_tracker: str = "bytetrack.yaml"
146     yolo_prefetch_workers: int = 2
147     min_action_window_duration: float = 2.0
148     # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
149     # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
150     # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
151     # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
152     # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
153     # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
154     # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
155     max_window_duration: float = 6.0
156     window_min_split_gap: float = 0.5 # min internal gap (s) that qualifies as a split
point
157     # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
158     # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
159     # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
160     # Only cuts (recall can't drop). OFF by default until measured/promoted.
161     window_hard_cap: bool = False
162     # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
163     # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
164     # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
165     # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
166     # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
167     # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
168     # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
169     window_inactivity_end: float = 1.5
170     inactivity_rally_thresh: float = 0.20
171     inactivity_min_density: float = 0.30
172     # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
173     # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
174     # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
175     # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
176     # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
177     # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
178     # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
179     # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
180     window_blob_fallback: bool = False

```

```

181         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
182         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
183         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
184         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
185         window_blob_factor: float = 4.0
186         log_yolo_candidates: bool = True
187         store_yolo_candidates: bool = True
188         ai_handoff_padding: float = 2.0
189         ai_max_workers: int = 5
190         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
191         # chunks of a high-bitrate (4K) source blow past the provider upload cap
192         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
193         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
194         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
195         # Matches gemini_refine's --clip-scale-width default.
196         ai_chunk_scale_width: int = 1280
197         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
198         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
199         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
200         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini

```

33. assistant (2026-06-22T09:43:00.801Z)

Perfect. Now let me verify a few more critical things about how the manifest is loaded and used:

34. user (2026-06-22T09:43:05.960Z)

```

Found 7 files
scripts\nightly_reelcount.py
docs\CODE_MAP.md
tools\generate_highlights.py
scripts\run_process_and_stitch.py
scripts\run_phase3_eval.py
backend\worker\_main_.py
backend\main.py

```

35. user (2026-06-22T09:43:06.216Z)

```

390:     for dotted, val in (manifest.get("config_overrides") or {}).items():

```

36. assistant (2026-06-22T09:43:08.540Z)

Now let me trace through the actual pipeline to check if the segmenter class registration works and returns the expected type:

37. user (2026-06-22T09:43:09.580Z)

```

1         """WASB-based hybrid play segmenter.
2
3         Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
4         WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5         badminton weights and runs today, so this is the live shuttle-motion substrate.
6
7         For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
8         candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9         min_window_duration) are what get tuned against golden rally labels.
10

```

```

11     The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12     lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13     WASB runner and identity.
14     """
15     from typing import Any, Dict, Tuple
16
17     from backend.pipeline.segmenters.base import segmenter_registry
18     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
19     from backend.pipeline.detectors.base import DetectorRunner
20     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
21
22
23     class WasbHybridSegmenter(TrajectoryHybridSegmenter):
24         family = "wasb"
25         display_label = "WASB"
26
27         @property
28         def name(self) -> str:
29             return "wasb_hybrid"
30
31         @property
32         def description(self) -> str:
33             return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
34                     "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
35
36         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
37             cfg = WasbConfig.from_indexing_cfg(idx_cfg)
38             return WasbRunner(cfg), {"weights_path": cfg.weights_path}
39
40         def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
41             # Lazy import: the native runner pulls no heavy deps, but keep parity with the
stub
42             # seam (imported only when detector_impl='native' is actually selected).
43             from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
NativeWasbConfig
44             cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
45             return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path,
46                                           "detector_impl": "native", "device": cfg.device}
47
48
49     segmenter_registry.register(WasbHybridSegmenter)
50

```

38. user (2026-06-22T09:43:10.328Z)

```

1     import cv2
2     import time
3     import random
4     from typing import List, Dict, Any, Tuple
5     import logging
6     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
7     from backend.results import Success, Failure # standardized contract
8
9     logger = logging.getLogger(__name__)
10
11     class MotionPlaySegmenter(BasePlaySegmenter):
12         @property
13         def name(self) -> str:
14             return "motion"
15
16         @property

```

```

17         def description(self) -> str:
18             return "Traditional CV pixel-motion heuristics tracking frame differences."
19
20         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
21             """
22             Processes a video to detect play segments using motion heuristics,
23             populates SQLite with detected segments, and indexes detailed events.
24             """
25             if not player_pool:
26                 player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]
27
28             cap = cv2.VideoCapture(video_path)
29             if not cap.isOpened():
30                 logger.error(f"Segmenter could not open video: {video_path}")
31                 return Failure(message=f"Could not open video: {video_path}",
is_recoverable=False)
32
33             final_segments = self._detect_motion_segments(cap, max_frames)
34
35             segments_data = self._populate_db_with_events(video_id, final_segments,
player_pool)
36
37             return Success(value=segments_data)
38
39         def _detect_motion_segments(self, cap: Any, max_frames: int = None) ->
List[Tuple[float, float]]:
40             """Run the pixel-motion heuristic over ``cap`` and return the detected
41             (start, end) play segments.
42
43             Reads/sub-samples frames, builds a per-frame motion signal, smooths it,
44             thresholds it into raw segments, then merges dead-time gaps and filters
45             out segments shorter than ``min_segment_duration``. ``cap`` is released
46             before returning. This is the detection half of ``process_video`` ? it
47             performs no DB writes and no fabrication.
48             """
49             fps = cap.get(cv2.CAP_PROP_FPS)
50             total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
51             duration = total_frames / fps if fps > 0 else 0
52
53             # Sub-sample settings (5 FPS for analysis)
54             analysis_fps = 5.0
55             frame_interval = max(1, int(fps / analysis_fps))
56
57             idx_cfg = self.config.get("indexing", {})
58             motion_threshold = idx_cfg.get("motion_threshold", 0.08)
59             min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
60             dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
61
62             prev_gray = None
63             motion_signals = []
64             timestamps = []
65
66             frame_idx = 0
67             processed_count = 0
68             t_start = time.time()
69             # Emit a progress line roughly every 5% of the video so long runs are
70             # observable instead of silent (a full 1080p match is ~tens of minutes).
71             progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
72             next_progress = progress_step
73             logger.info(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
74                 f"(~{duration/60:.1f} min), analyzing every {frame_interval}th
frame")
75             while True:
76                 # Skip decoding for intermediate frames using cap.grab() for high

```

```

performance
77         for _ in range(frame_interval - 1):
78             if not cap.grab():
79                 break
80             frame_idx += 1

```

39. assistant (2026-06-22T09:43:12.376Z)

Now let me check trajectory_hybrid to understand the actual process_video contract:

40. user (2026-06-22T09:43:13.922Z)

```

1      """Shared engine for trajectory-substrate hybrid segmenters.
2
3      tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4      standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5      now the single home for the whole pipeline shape:
6
7          healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8          -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10     A concrete subclass supplies only its identity and runner wiring:
11     - ``family``          ? config section under ``indexing.*``, output subdir, and
12                           the ``<family>-hybrid-<model>`` detector tag in metadata.
13     - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14     - ``name`` / ``description`` properties (the registry contract).
15     - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16                                     detection_params extras such as weights_path).
17
18     This base is deliberately NOT registered; only concrete subclasses register.
19     """
20     import os
21     import time
22     import logging
23     import concurrent.futures
24     from typing import Any, Dict, List, Optional, Tuple
25
26     import cv2
27
28     from backend.pipeline.segmenters.base import BasePlaySegmenter
29     from backend.utils.video import get_video_duration, extract_video_chunk
30     from backend.utils.paths import output_base
31     from backend.sports import sport_registry
32     from backend.providers import provider_registry
33     from backend.pipeline.detectors.base import DetectorRunner
34
35     logger = logging.getLogger(__name__)
36
37
38     def _fmt_ts(seconds: float) -> str:
39         seconds = max(0, int(seconds))
40         h, rem = divmod(seconds, 3600)
41         m, s = divmod(rem, 60)
42         return f"{h:02d}:{m:02d}:{s:02d}"
43
44
45     class TrajectoryHybridSegmenter(BasePlaySegmenter):
46         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
47         boundaries."""
48
49         #: config section under `indexing` (velocity_threshold lives there), the
50         #: output subdir for trajectories, and the metadata detector-tag prefix.
51         family: str = ""
52         #: human-readable label used in log lines.
53         display_label: str = ""

```

```

53
54     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
55         """Return (runner, detector-specific detection_params extras) for the default
(WSL) path."""
56         raise NotImplementedError
57
58     def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
59         """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
with a
60         native runner override this; the base refuses with a clear message (M3.5: WASB
only)."""
61         raise NotImplementedError(
62             f"detector_impl='native' is not supported for the "
63             f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
has a "
64             f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
65
66     def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
67         """Resolve the detector runner, honouring the CPU-stub and Linux-native
overrides.
68
69         ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
``indexing.detector_impl``:
70         ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
pipeline
71         is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
72         in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
73         ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
74         (the WSL runner) so the default production path is unchanged
75         (``docs/DECOUPLED_COMPUTE/``).
76         """
77         impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
78         if impl == "stub":
79             from backend.pipeline.detectors.stub_runner import StubDetectorRunner,
StubConfig
80             logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
self.display_label or "trajectory")
81             return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)),
{"detector_impl": "stub"}
82         if impl in ("native", "native_wasb", "wasb_native"):
83             logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
self.display_label or "trajectory")
84             return self._build_native_runner(idx_cfg)
85         return self._build_runner(idx_cfg)
86
87     @staticmethod
88     def _probe_fps_width(video_path: str) -> tuple:
89         """Return (fps, frame_width) for a video, with sensible fallbacks."""
90         cap = cv2.VideoCapture(video_path)
91         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
92         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
93         cap.release()
94         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
95
96     @staticmethod
97     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
98         """Provider-reported exchange count, else a duration-based estimate.
99
100         One canonical implementation ? the twins had drifted (wasb relied on
101
102

```

```

103         ``int(None)`` raising into the fallback; same result, by accident).
104         """
105         shot_count = segment.get("shuttle_exchange_count")
106         if shot_count is None:
107             return max(3, int(duration / 0.8))
108         try:
109             return int(shot_count)
110         except (ValueError, TypeError):
111             return max(3, int(duration / 0.8))
112
113     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
114     -----
115     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
116                                frame_width: float, video_path: str,
117                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
118         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
119         BASE
120         returns exactly the 4 motion keys, so the trajectory twins persist byte-
121         identical
122         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
123         features). ``points`` are the whole-video trajectory points
124         (TrajectoryPoint)."""
125         return {
126             "peak_velocity": cw["peak_velocity"],
127             "mean_velocity": cw["mean_velocity"],
128             "active_frame_count": cw["active_frame_count"],
129             "track_id_count": cw["track_id_count"],
130         }
131
132     def _select_for_handoff(self, windows: List[Dict[str, Any]],
133                            window_stats: List[Dict[str, Any]],
134                            idx_cfg: Dict[str, Any]) -> List[int]:
135         """Indices of the candidate windows that proceed to the AI handoff (and thus may
136         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
137         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
138         Persisted candidates are NOT affected ? they remain the full superset for
139         offline
140         re-scoring."""
141         return list(range(len(windows)))
142
143     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
144                      player_pool: Optional[List[str]] = None,
145                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
146         # override-ignore: the ABC declares the Result contract (Success|Failure)
147         # but every live segmenter still returns a bare list ? the documented
148         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
149         label = self.display_label
150         # --- Sport profile + AI provider wiring (identical across segmenters) ---
151         sport_name = self.config.get("sport", "badminton")
152         try:
153             sport_profile = sport_registry.get(sport_name)
154         except KeyError as e:
155             logger.error(str(e))

```

41. assistant (2026-06-22T09:43:16.039Z)

Let me check the rest of the process_video method:

42. user (2026-06-22T09:43:17.566Z)

```

140         max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
141         # override-ignore: the ABC declares the Result contract (Success|Failure)
142         # but every live segmenter still returns a bare list ? the documented
143         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
144         label = self.display_label

```

```

145         # --- Sport profile + AI provider wiring (identical across segmenters) ---
146         sport_name = self.config.get("sport", "badminton")
147         try:
148             sport_profile = sport_registry.get(sport_name)
149         except KeyError as e:
150             logger.error(str(e))
151             return []
152
153         idx_cfg = self.config.get("indexing", {})
154         # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
155         the
156         # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
157         is
158         # needed and nothing is uploaded. Used to run the local detector standalone
159         (e.g. to
160         # measure it against the Gemini path, or to avoid the cloud entirely). With
161         # FusionSegmenter the windows are still Gate-A/B-filtered via
162         `_select_for_handoff`.
163         skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
164         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
165         "gemini"))
166         if skip_ai:
167             model_name = "local-noai"
168             logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
169         windows will "
170         "be emitted as rallies; no %s handoff, no API key needed.",
171         label, provider_name)
172         else:
173             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
174         "gemini-2.5-flash"))
175             api_key_env_var = f"{provider_name.upper()}_API_KEY"
176             api_key = os.environ.get(api_key_env_var)
177             if not api_key:
178                 from backend.pipeline.segmenters._keyhint import api_key_hint
179                 logger.error(
180                     f"{api_key_env_var} environment variable is not set! "
181                     f"To run the {provider_name} handoff, set your API key. "
182                     + api_key_hint(api_key_env_var)
183                 )
184             return []
185         try:
186             provider = provider_registry.get(provider_name, api_key=api_key,
187         model_name=model_name)
188         except KeyError as e:
189             logger.error(str(e))
190             return []
191
192         video_duration = get_video_duration(video_path)
193         if video_duration <= 0:
194             logger.error("Invalid video duration.")
195             return []
196         fps, frame_width = self._probe_fps_width(video_path)
197
198         # --- Runner config + windowing thresholds ---
199         # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
200         # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
201         # it delegates to the subclass _build_runner (the real WSL/native runner).
202         try:
203             runner, runner_params = self._resolve_runner(idx_cfg)
204         except NotImplementedError as e:
205             # e.g. detector_impl="native" for a family without a native runner
206             (tracknet, or
207             # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of
208             crashing.
209             logger.error("%s: %s", label, e)

```



```

200         return []
201
202     velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
203     merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
204     min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
205     # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
206     max_window_duration = idx_cfg.get("max_window_duration", 0.0)
207     window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
208     window_hard_cap = idx_cfg.get("window_hard_cap", False)
209     window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
210     inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
211     inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
212     window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
213     window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
214     handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
215     ai_max_workers = idx_cfg.get("ai_max_workers", 5)
216     log_candidates = idx_cfg.get("log_yolo_candidates", True)
217     store_candidates = idx_cfg.get("store_yolo_candidates", True)
218
219     detection_params = {
220         "detector": self.name,
221         "velocity_thresh": velocity_thresh,
222         "merge_gap": merge_gap,
223         "min_action_window_duration": min_window_duration,
224         **runner_params,
225     }
226
227     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
228     ok, msg = runner.healthcheck()
229     if not ok:
230         logger.error("%s detector environment not ready: %s", label, msg)
231         return []
232     logger.info("%s detector env OK: %s", label, msg)
233
234     # --- Phase 2: trajectory -> candidate rally windows ---
235     # Trajectory detectors process the whole video in one pass (they carry
236     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
237     t_start = time.time()
238     out_dir = os.path.join(output_base(self.config), self.family)
239     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
240     if not csv_path:
241         logger.error("%s produced no trajectory; aborting.", label)
242         return []
243
244     points = runner.parse_trajectory_csv(csv_path)
245     logger.info("Parsed %d trajectory points (%d visible).",
246               len(points), sum(1 for p in points if p.visible))
247
248     all_candidate_windows = runner.trajectory_to_action_windows(
249         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
250         velocity_thresh=velocity_thresh, merge_gap=merge_gap,
251         min_window_duration=min_window_duration, chunk_index=0,
252         max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
253         window_hard_cap=window_hard_cap,
254         window_inactivity_end=window_inactivity_end,
255         inactivity_rally_thresh=inactivity_rally_thresh,
256         inactivity_min_density=inactivity_min_density,
257         window_blob_fallback=window_blob_fallback,
258         window_blob_factor=window_blob_factor,
259     )
260     logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
261               label, time.time() - t_start, len(all_candidate_windows))
262
263     if log_candidates:

```

```

264         for cw in all_candidate_windows:
265             logger.info(f"[{label}] rally]
{ _fmt_ts(cw['start'])}-{ _fmt_ts(cw['end'])} "
266             f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
267             f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")
268
269         # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
270         # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
271         # persistence and the handoff gate below.
272         window_stats = [
273             self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
idx_cfg)
274             for cw in all_candidate_windows
275         ]
276
277         # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
278         candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
279         if store_candidates:
280             for i, cw in enumerate(all_candidate_windows):
281                 candidate_ids[i] = self.db.add_candidate_segment(
282                     video_id, detector=self.name,
283                     start_time=cw["start"], end_time=cw["end"],
284                     chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
285                     stats=window_stats[i], detection_params=detection_params,
286                 )
287
288         if not all_candidate_windows:
289             return []
290
291         # --- Phase 3: AI provider handoff over padded candidate clips ---
292         # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
293         # candidates above stay the full superset ? only the served play_segments are
gated.
294         handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
295
296         ai_segments: List[dict] = []
297         if skip_ai:
298             # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
299             # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
300             # falls back to the duration-based estimate (no AI exchange count).
301             ai_segments = [
302                 {
303                     "start_time": all_candidate_windows[wi]["start"],
304                     "end_time": all_candidate_windows[wi]["end"],
305                     "shuttle_exchange_count": None,
306                     "shot_count_confidence": "LocalNoAI",
307                     "_candidate_id": candidate_ids[wi],
308                 }
309                 for wi in handoff_indices
310             ]
311             logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
312                         "(no AI handoff).", len(ai_segments))
313         else:
314             handoff_dir = os.path.join(output_base(self.config),
f"chunks_{provider_name}_handoff")

```

```

315         os.makedirs(handoff_dir, exist_ok=True)
316         logger.info("Phase 3: %s validation on %d candidate windows...",
317                     provider_name, len(handoff_indices))
318
319     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
320         w_start = max(0, window["start"] - handoff_padding)
321         w_end = min(video_duration, window["end"] + handoff_padding)
322         w_dur = w_end - w_start
323         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
324         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
325             return []
326         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
327         segments, _ = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur,
328                                             label=f"Handoff {wi+1}")
329         valid = []
330         for seg in segments:
331             try:
332                 seg["start_time"] = float(seg["start_time"]) + w_start
333                 seg["end_time"] = float(seg["end_time"]) + w_start
334                 seg["_candidate_id"] = candidate_ids[wi]
335                 valid.append(seg)
336             except (ValueError, TypeError, KeyError) as e:
337                 logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
338             try:
339                 os.remove(w_path)
340             except OSError:
341                 pass
342         return valid
343
344     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
345         futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
346                     for wi in handoff_indices]
347         for future in futures:
348             ai_segments.extend(future.result())
349         try:
350             os.rmdir(handoff_dir)
351         except OSError:
352             pass
353         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
354
355     # --- Phase 4: DB population + candidate linking ---
356     segments_data = []
357     for segment in ai_segments:
358         try:
359             start = float(segment["start_time"])
360             end = float(segment["end_time"])
361             dur = end - start
362             metadata = {
363                 "shot_count": self._shot_count_from(segment, dur),
364                 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
365                 "detector": f"{self.family}-hybrid-{model_name}",
366             }
367             segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
368             if segment_id:
369                 candidate_id = segment.get("_candidate_id")
370                 if candidate_id is not None:
371                     self.db.link_candidate_to_segment(candidate_id, segment_id)

```

```

372             segments_data.append({
373                 "id": segment_id, "start_time": start, "end_time": end,
374                 "duration": dur, "metadata": metadata,
375             })
376         except Exception as e:
377             logger.warning("Skipping invalid segment struct: %s", e)
378
379     return segments_data
380

```

43. assistant (2026-06-22T09:43:20.250Z)

Perfect! Now I have a clear picture. Let me verify the schema of returned segments and check one more critical detail about how the nightly runner handles the return value:

44. user (2026-06-22T09:43:22.574Z)

```

344-         t0 = time.monotonic()
345-         result = seg_cls(db, config).process_video(video_id, local)
346-         wall = time.monotonic() - t0
347-         if isinstance(result, Failure):
348-             return None, None, wall, getattr(result, "message", "segmenter failed")
349-         segs = result.value if hasattr(result, "value") else result
350-         intervals = _segments_to_intervals(segs)
351-         return len(segs), intervals, wall, None
352-

```

45. assistant (2026-06-22T09:43:26.305Z)

Excellent. Now I understand the full contract. Let me trace through one more critical area?the actual issue: the returned segments from trajectory_hybrid have `start\_time`/`end\_time` keys (line 372-375 in trajectory_hybrid.py), and the nightly runner correctly handles both `start\_time`/`end\_time` and `start`/`end` in `\_segments\_to\_intervals`. Let me verify the billing module one more time:

46. user (2026-06-22T09:43:26.929Z)

```

1     """Per-job cost ledger ? pure cost math (COMPUTE_DECOUPLED_SERVING M8, decision D8).
2
3     ``cost_usd(job) = ?_resource usage(job, resource) × rate(resource, pricebook_version)``
4     (the formula from the archived 05-COST-ATTRIBUTION design). Cost is computed + stored in
**USD**
5     (matches GCP billing = one source of truth) and **displayed in INR** at a configurable
FX rate
6     (the Bangalore market). The pricebook is a versioned DB table
(``backend/infrastructure/database.py``);
7     this module is the pure arithmetic over a resolved ``{resource: unit_price_usd}`` rates
dict, so it
8     is unit-tested with zero IO and the DB layer owns the version/gpu_type lookup.
9
10    **Default-OFF:** with the seeded ``placeholder-v0`` pricebook (every rate ``0.0``) every
cost is
11    ``0.0`` ? nothing user-facing changes until the owner inserts a real, calibrated
pricebook version.
12    """
13
14    from __future__ import annotations
15
16    from typing import Dict, Mapping, Tuple
17
18    # Metered resource keys ? shared by the usage dict, the pricebook rows, and the
breakdown.
19    GPU_SECONDS = "gpu_seconds"      # GPU-active seconds (the big one) ? rate is per-second,
per gpu_type

```

```

20     WALL_SECONDS = "wall_seconds"    # total wall time (separate from GPU so idle/sync isn't
billed as GPU)
21     EGRESS_GB = "egress_gb"          # reel/byte egress in GB
22     GEMINI_USD = "gemini_usd"        # provider-reported cost, ALREADY in USD (a passthrough,
not rate-×)
23
24     # Resources whose cost = amount × unit_price (GEMINI_USD is excluded ? it is already a
USD amount).
25     _RATE_MULTIPLIED = (GPU_SECONDS, WALL_SECONDS, EGRESS_GB)
26
27
28     def compute_cost_usd(usage: Mapping[str, float],
29                          rates: Mapping[str, float]) -> Tuple[float, Dict[str, float]]:
30         """Return ``(cost_usd, breakdown)`` for one job.
31
32         ``usage`` maps a resource key to its measured amount (seconds / GB); ``rates`` maps
a resource
33         key to its USD unit price (already resolved for this pricebook version + gpu_type by
the DB
34         layer). ``GEMINI_USD`` in ``usage`` is added verbatim (it is already a USD cost, not
multiplied).
35         Missing amounts/rates default to ``0.0`` ? with the placeholder pricebook the total
is ``0.0``.
36         """
37         breakdown: Dict[str, float] = {}
38         total = 0.0
39         for res in _RATE_MULTIPLIED:
40             amount = float(usage.get(res, 0.0) or 0.0)
41             rate = float(rates.get(res, 0.0) or 0.0)
42             cost = round(amount * rate, 6)    # round the LINE, then sum rounded lines so
the
43             if amount or rate:                # breakdown reconciles to the total exactly
(no drift),
44             breakdown[res] = cost            # and record the line even at $0 for
transparency
45             total += cost
46         gemini = round(float(usage.get(GEMINI_USD, 0.0) or 0.0), 6)
47         if gemini:
48             breakdown[GEMINI_USD] = gemini
49         total += gemini
50         return round(total, 6), breakdown
51
52
53     def apply_margin(cost_usd: float, margin: float) -> float:
54         """Resale price = ``cost_usd × (1 + margin)``. ``margin`` is clamped ? 0 (never
below cost)."""
55         return round(cost_usd * (1.0 + max(0.0, float(margin))), 6)
56
57
58     def to_inr(cost_usd: float, fx_inr_per_usd: float) -> float:
59         """Display conversion: USD ? INR at the configured FX rate (2 dp, the display
currency)."""
60         return round(float(cost_usd) * float(fx_inr_per_usd), 2)
61

```

47. assistant (2026-06-22T09:43:29.503Z)

Perfect! Now let me check one final detail about the cost_report function call:

48. user (2026-06-22T09:43:31.494Z)

```

68-
69-
70-# ----- #
71-# Pure cost math (wraps backend.billing ? no IO).

```

```

72-# ----- #
73: def cost_report(billed_seconds: float, machine_usd_per_hr: float,
74-                fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
75-     """Run cost from billed wall-seconds x machine rate ? USD + INR (`backend.billing`
arithmetic).
76-
77-     ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes ``--vm-
uptime-seconds``;
78-     otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays for
boot/install)."""
79-     rate_per_sec = float(machine_usd_per_hr) / 3600.0
80:     usage = {billing.WALL_SECONDS: float(billed_seconds)}
81:     rates = {billing.WALL_SECONDS: rate_per_sec}
82:     usd, breakdown = billing.compute_cost_usd(usage, rates)
83:     return {
84-         "billed_seconds": round(float(billed_seconds), 1),
85-         "gpu_seconds": round(float(gpu_seconds), 1),
86-         "machine_usd_per_hr": float(machine_usd_per_hr),

```

49. assistant (2026-06-22T09:43:33.691Z)

Now let me check the actual returned segments schema to be absolutely sure the key names are consistent:

50. user (2026-06-22T09:43:35.469Z)

```

281-         candidate_ids[i] = self.db.add_candidate_segment(
282-             video_id, detector=self.name,
283-             start_time=cw["start"], end_time=cw["end"],
284-             chunk_index=cw["chunk_index"], confidence=min(1.0, cw["peak_velocity"]),
285-             stats=window_stats[i], detection_params=detection_params,
--
302-         {
303-             "start_time": all_candidate_windows[wi]["start"],
304-             "end_time": all_candidate_windows[wi]["end"],
305-             "shuttle_exchange_count": None,
306-             "shot_count_confidence": "LocalNoAI",
--
331-             try:
332-                 seg["start_time"] = float(seg["start_time"]) + w_start
333-                 seg["end_time"] = float(seg["end_time"]) + w_start
334-                 seg["_candidate_id"] = candidate_ids[wi]
335-                 valid.append(seg)
--
358-         try:
359-             start = float(segment["start_time"])
360-             end = float(segment["end_time"])
361-             dur = end - start
362-             metadata = {
--
370-                 if candidate_id is not None:
371-                     self.db.link_candidate_to_segment(candidate_id, segment_id)
372-             segments_data.append({
373-                 "id": segment_id, "start_time": start, "end_time": end,
374-                 "duration": dur, "metadata": metadata,
375-             })

```

51. assistant (2026-06-22T09:43:37.968Z)

Perfect! Now let me verify one more thing?double-check the config.indexing structure to see if the dotted override path is valid:

52. user (2026-06-22T09:43:39.569Z)

```

67-     wsl_stage_dir: str = "~/clips"
68-     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69-     velocity_threshold: float = 0.02
70-     # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner. ---
71-     # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
72-     # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4 CPU
fallback
73-     # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-CPU
downgrade).
--
124-     net_position: float = Field(default=0.5, ge=0.0, le=1.0)
125-
126-
127-class IndexingConfig(BaseModel):
128-     default_segmenter: str = "yolo_hybrid"
129-     use_gpu: bool = True
130-     # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
131-     # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
132-     # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
133-     # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
134-     # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
135-     detector_impl: str = "auto"
136-     motion_threshold: float = 0.08
137-     min_segment_duration: float = 3.0
138-     dead_time_merge_gap: float = 2.0
--
207-     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
208-     fusion: FusionConfig = Field(default_factory=FusionConfig)
209-
210-     @field_validator("default_segmenter")
211-     @classmethod
212-     def segmenter_must_be_registered(cls, v: str) -> str:
213-         # Registry check happens at runtime in main/segmenter selection.
--
354-     ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
355-     *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core reads
356-     these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
357-     (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the loader.
358-
359-     Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
360-

```

53. assistant (2026-06-22T09:43:41.810Z)

Perfect! All the pieces check out. Now let me do a final comprehensive check for any potential API mismatches by examining how the config is actually used in the runner:

54. user (2026-06-22T09:43:43.529Z)

```

387-
388-     config = load_config()
389-     # apply manifest config overrides onto the typed config (dotted keys)
390-     for dotted, val in (manifest.get("config_overrides") or {}).items():
391-         _set_dotted(config, dotted, val)
392-     segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
"default_segmenter", "motion")
393-     b_pv = (baseline or {}).get("per_video", {})
394-     results: List[Dict[str, Any]] = []
395-     total_wall = 0.0
396-     for v in manifest.get("videos", []):
397-         base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
--
496-         billed_seconds=billed,

```

```

497-         machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
DEFAULT_MACHINE_USD_PER_HR)),
498-         fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
499-         gpu_seconds=total_wall,
500-     )
501:     segmenter = manifest.get("segmenter", "motion")
502-     snapshot = summarize_results(run_results, segmenter, cost)
503-     snapshot["git_commit"] = _git_head()
504-     snapshot["generated_at"] = date
505-
506-     if args.update_baseline:

```

55. assistant (2026-06-22T09:43:45.728Z)

Now let me check the manifest once more and verify all the videos in it have the required fields:

56. user (2026-06-22T09:43:46.591Z)

```

1      {
2      "_doc": "Nightly E2E engine reel-count manifest (scripts/nightly_reelcount.py). The
configured+checkpointed pipeline runs on each video on a GCP GPU VM; the reel count + quality
metrics are asserted vs eval_baselines/reelcount_baseline.json. ADD A VIDEO = append one row +
re-freeze: `python scripts/nightly_reelcount.py --update-baseline` (commit the updated
baseline). URIs may be gs:// (recommended, co-located with the VM), gdrive://, or local paths.
labels_uri (optional) is a golden-features JSON (gts key) or a start,end CSV ? present =>
quality metrics (tol_f1/f1@0.5/f1@0.25) are scored too. EDIT the example rows below to your
actual short (~5-6 min) OWNED, minors-free clips before enabling.",
3      "segmenter": "wasb_hybrid",
4      "detector_impl": "native",
5      "config_overrides": {
6      "indexing.default_segmenter": "wasb_hybrid",
7      "indexing.detector_impl": "native"
8      },
9      "machine_usd_per_hr": 0.70,
10     "fx_inr_per_usd": 83.0,
11     "videos": [
12     {
13         "name": "GX030094",
14         "uri": "gs://khealsutra-rally-corpus/nightly/videos/GX030094.mp4",
15         "labels_uri": "gs://khealsutra-rally-
corpus/nightly/labels/GX030094_golden_features.json"
16     },
17     {
18         "name": "Badminton_BXH_2",
19         "uri": "gs://khealsutra-rally-corpus/nightly/videos/Badminton_BXH_2.mp4",
20         "labels_uri": "gs://khealsutra-rally-
corpus/nightly/labels/Badminton_BXH_2_golden_features.json"
21     },
22     {
23         "name": "GX010128",
24         "uri": "gs://khealsutra-rally-corpus/nightly/videos/GX010128.mp4",
25         "labels_uri": "gs://khealsutra-rally-
corpus/nightly/labels/GX010128_golden_features.json"
26     }
27     ]
28     }
29

```

57. assistant (2026-06-22T09:43:49.358Z)

Now let me do one final check?verify the exact return type contract by checking the unwrap logic once more:

58. user (2026-06-22T09:43:50.452Z)

```
340         def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]], float, Optional[str]]:
341             random.seed(0) # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
342             db = Database(db_path)
343             video_id = compute_video_id(local)
344             t0 = time.monotonic()
345             result = seg_cls(db, config).process_video(video_id, local)
346             wall = time.monotonic() - t0
347             if isinstance(result, Failure):
348                 return None, None, wall, getattr(result, "message", "segmenter failed")
349             segs = result.value if hasattr(result, "value") else result
350             intervals = _segments_to_intervals(segs)
351             return len(segs), intervals, wall, None
352
353         import tempfile
354         total_wall = 0.0
355         with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
356             count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
357             total_wall += wall
358             if err is not None:
359                 return {"name": name, "reel_count": None, "ok": False, "error": err,
```

59. assistant (2026-06-22T09:43:53.157Z)

Perfect! Now let me examine one potential gotcha?I want to verify the exact flow when storing return values from the segmenter. Let me trace through the motion segmenter to see if it actually returns a Success or a bare list:

60. user (2026-06-22T09:43:54.321Z)

```
15
16         @property
17         def description(self) -> str:
18             return "Traditional CV pixel-motion heuristics tracking frame differences."
19
20         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
21             """
22             Processes a video to detect play segments using motion heuristics,
23             populates SQLite with detected segments, and indexes detailed events.
24             """
25             if not player_pool:
26                 player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]
27
28             cap = cv2.VideoCapture(video_path)
29             if not cap.isOpened():
30                 logger.error(f"Segmenter could not open video: {video_path}")
31                 return Failure(message=f"Could not open video: {video_path}",
is_recoverable=False)
32
33             final_segments = self._detect_motion_segments(cap, max_frames)
34
35             segments_data = self._populate_db_with_events(video_id, final_segments,
player_pool)
36
37             return Success(value=segments_data)
38
39         def _detect_motion_segments(self, cap: Any, max_frames: int = None) ->
List[Tuple[float, float]]:
40             """Run the pixel-motion heuristic over ``cap`` and return the detected
41             (start, end) play segments.
42
```

```

43 Reads/sub-samples frames, builds a per-frame motion signal, smooths it,
44 thresholds it into raw segments, then merges dead-time gaps and filters
45 out segments shorter than ``min_segment_duration``. ``cap`` is released
46 before returning. This is the detection half of ``process_video`` ? it
47 performs no DB writes and no fabrication.
48 """
49 fps = cap.get(cv2.CAP_PROP_FPS)
50 total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
51 duration = total_frames / fps if fps > 0 else 0
52
53 # Sub-sample settings (5 FPS for analysis)
54 analysis_fps = 5.0
55 frame_interval = max(1, int(fps / analysis_fps))
56
57 idx_cfg = self.config.get("indexing", {})
58 motion_threshold = idx_cfg.get("motion_threshold", 0.08)
59 min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
60 dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
61
62 prev_gray = None
63 motion_signals = []
64 timestamps = []
65
66 frame_idx = 0
67 processed_count = 0
68 t_start = time.time()
69 # Emit a progress line roughly every 5% of the video so long runs are
70 # observable instead of silent (a full 1080p match is ~tens of minutes).
71 progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
72 next_progress = progress_step
73 logger.info(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
74            f"(~{duration/60:.1f} min), analyzing every {frame_interval}th
frame")
75 while True:
76     # Skip decoding for intermediate frames using cap.grab() for high
performance
77     for _ in range(frame_interval - 1):
78         if not cap.grab():
79             break
80         frame_idx += 1
81
82     ret, frame = cap.read()
83     if not ret:
84         break
85
86     # Process frame
87     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
88     # Apply Gaussian Blur to smooth noise
89     gray = cv2.GaussianBlur(gray, (21, 21), 0)
90
91     t = frame_idx / fps
92     timestamps.append(t)
93
94     if prev_gray is not None:
95         # Frame absolute difference (highly optimized motion tracking)
96         diff = cv2.absdiff(prev_gray, gray)
97         _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
98
99         # Dilate to fill gaps
100         thresh = cv2.dilate(thresh, None, iterations=2)
101
102         # Motion ratio
103         motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
104         motion_signals.append(motion_ratio)

```

```

105         else:
106             motion_signals.append(0.0)
107
108         prev_gray = gray
109         frame_idx += 1
110         processed_count += 1
111
112         if progress_step and frame_idx >= next_progress:
113             pct = 100.0 * frame_idx / total_frames
114             elapsed = time.time() - t_start
115             eta = elapsed / (frame_idx / total_frames) - elapsed if frame_idx else 0
116             logger.info(f"[motion] {pct:5.1f}% ? {frame_idx}/{total_frames} frames,
117 "
118                         f"{processed_count} analyzed, {elapsed:.0f}s elapsed,
119 ~{eta:.0f}s left")
120             next_progress += progress_step
121
122         if max_frames is not None and processed_count >= max_frames:
123             logger.debug(f"Reached max_frames limit of {max_frames}. Halting indexer
124 frame parsing.")
125             break
126
127         if frame_idx >= total_frames:
128             break
129
130         cap.release()
131
132         # Smooth signal using a moving average
133         window_size = 5
134         smoothed = []
135         for i in range(len(motion_signals)):
136             start = max(0, i - window_size // 2)
137             end = min(len(motion_signals), i + window_size // 2 + 1)
138             smoothed.append(sum(motion_signals[start:end]) / (end - start))
139
140         # Detect active play boundaries
141         in_segment = False
142         segment_start = None
143         raw_segments: List[Tuple[float, float]] = []
144
145         for i, t in enumerate(timestamps):
146             signal = smoothed[i]
147             if signal > motion_threshold:
148                 if not in_segment:
149                     in_segment = True
150                     segment_start = t
151             else:
152                 if in_segment:
153                     in_segment = False
154                     raw_segments.append((segment_start, t))
155
156         # Handle video ending while in active segment
157         if in_segment:
158             raw_segments.append((segment_start, timestamps[-1]))
159
160         # Post-Processing: 1. Merge segments separated by less than dead_time_merge_gap
161         merged_segments: List[Tuple[float, float]] = []
162         if raw_segments:
163             curr_start, curr_end = raw_segments[0]
164             for next_start, next_end in raw_segments[1:]:
165                 if next_start - curr_end < dead_time_merge_gap:
166                     # Merge
167                     curr_end = next_end
168             else:
169                 merged_segments.append((curr_start, curr_end))

```

```

167         curr_start, curr_end = next_start, next_end
168         merged_segments.append((curr_start, curr_end))
169
170     # Post-Processing: 2. Filter segments shorter than min_segment_duration
171     final_segments: List[Tuple[float, float]] = [
172         r for r in merged_segments if (r[1] - r[0]) >= min_segment_duration
173     ]
174
175     return final_segments
176
177     def _populate_db_with_events(self, video_id: str, final_segments: List[Tuple[float,
float]],
178                                player_pool: List[str]) -> List[Dict[str, Any]]:
179         """Persist ``final_segments`` and their fabricated metadata/events to the DB.
180
181         For each detected segment this assigns a random server/receiver, fabricates
182         rich tags, then indexes a serve / 0?3 smash / termination event set ? all
183         with the stdlib ``random`` module (the legacy demo's fabricated baseline).
184         Returns the per-segment dicts handed back to the caller. The ``random``
185         call order here is load-bearing for reproducibility and must not change.
186         """
187         # Populate SQLite DB with segments and metadata events
188         segments_data = []
189         for start, end in final_segments:
190             # Assign Server and Receiver randomly from pool
191             server, receiver = random.sample(player_pool, 2)
192
193             # Formulate dynamic rich tags
194             dur = end - start
195             metadata = {
196                 "shot_count": random.randint(4, 25),
197                 "intensity": "high" if dur > 12 else "medium",
198                 "avg_speed_kmh": round(random.uniform(40, 85), 1)
199             }
200
201             segment_id = self.db.add_play_segment(video_id, start, end, server,
receiver, metadata)
202
203             if segment_id:
204                 # Add sub-segment events
205                 # A: Serve Event (always at start)
206                 serve_foul = random.random() < 0.15 # 15% chance of serve foul
207                 self.db.add_event(
208                     segment_id=segment_id,
209                     timestamp=start + 0.5,
210                     event_type="serve",
211                     player=server,
212                     details={"status": "foul" if serve_foul else "valid", "type":
"low_serve" if random.random() > 0.5 else "high_serve"}
213                 )
214

```

61. assistant (2026-06-22T09:43:56.540Z)

Let me continue reading the motion segmenter to see the complete return:

62. user (2026-06-22T09:43:58.198Z)

```

200
201         segment_id = self.db.add_play_segment(video_id, start, end, server,
receiver, metadata)
202
203         if segment_id:
204             # Add sub-segment events
205             # A: Serve Event (always at start)

```

```

206         serve_foul = random.random() < 0.15 # 15% chance of serve foul
207         self.db.add_event(
208             segment_id=segment_id,
209             timestamp=start + 0.5,
210             event_type="serve",
211             player=server,
212             details={"status": "foul" if serve_foul else "valid", "type":
"low_serve" if random.random() > 0.5 else "high_serve"}
213         )
214
215         # B: Smash Events (1 to 3 random smashes per segment)
216         smashes_count = random.randint(0, 3)
217         for _ in range(smashes_count):
218             smash_time = random.uniform(start + 1.5, end - 1.0)
219             smasher = random.choice([server, receiver])
220             self.db.add_event(
221                 segment_id=segment_id,
222                 timestamp=round(smash_time, 2),
223                 event_type="smash",
224                 player=smasher,
225                 details={"speed_kmh": round(random.uniform(120, 250), 1)}
226             )
227
228         # C: Segment Termination Event (foul/winner)
229         foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
230         is_foul = random.random() > 0.4
231         error_player = random.choice([server, receiver])
232
233         self.db.add_event(
234             segment_id=segment_id,
235             timestamp=end - 0.2,
236             event_type="foul" if is_foul else "winner",
237             player=error_player,
238             details={"type": random.choice(foul_types)} if is_foul else
{"shot_type": "drop" if random.random() > 0.5 else "smash"}
239         )
240
241         # Store segment representation to return to caller
242         segments_data.append({
243             "id": segment_id,
244             "start_time": start,
245             "end_time": end,
246             "duration": dur,
247             "server": server,
248             "receiver": receiver,
249             "metadata": metadata
250         })
251
252         return segments_data
253
254     # Register the segmenter
255     segmenter_registry.register(MotionPlaySegmenter)
256

```

63. assistant (2026-06-22T09:44:00.793Z)

Excellent! Now I have comprehensive understanding. Let me now compile my findings. I've thoroughly reviewed the code and found several critical issues. Let me create the structured output:

64. user (2026-06-22T09:44:55.785Z)

Structured output provided successfully

Summary

I've completed a thorough integration faithfulness review of the nightly regression change. The analysis checked the runner against actual backend implementation across five key areas:

Critical findings (2 high-severity, documentation-only):

1. **motion.py type-hint debt** ? The return type annotation says `List[Dict]` but the code returns `Success | Failure`. This is a lie in the type system, but the nightly runner handles both cases correctly at line 349 via `.value` fallback. Runtime is safe; only the type hint is stale.

2. **trajectory_hybrid.py documented-gap** ? The code comment at line 141-143 `_admits_` that `trajectory_hybrid` returns a bare list, not `Result[List]`, violating the base class contract. This is called a "deliberate-incremental gap" in `CODE_MAP` (a known debt). The nightly runner masks it correctly.

All other verifications passed ?:

- `Segmenter` registry API (class ? instance) is correctly called
- `Result[List[Dict]]` unwrapping handles both `Success` wrappers and bare lists
- `start_time`/`end_time`` keys match; runner is defensive with dual-key fallback
- `backend.storage.fetch()` handles `gs://` URIs correctly
- `rally_seg_eval.score_intervals()` returns the expected metric keys
- `load_config()` + dotted overrides target real `IndexingConfig` fields
- `Database(db_path)` construction and reel-count via `len(segs)` are sound
- `billing.WALL_SECONDS` constant exists and cost math is correct

No production bugs found ? the runner will work correctly on the VM. The type-system gaps are pre-existing deliberate debts, not new regressions introduced by this PR. The code is defensive and handles edge cases properly.